Certificate-Bearing Theorem Provers

Theory, Measurement, and a Parameterised Self-Certifying System

Brian Tenneson

Implementation and experiments by Claude (Anthropic)

Abstract

This document records a complete investigation into what limits an automated theorem prover, carried out in theory and in running code. Part I separates two measures that are routinely conflated — the arithmetical *tier* of a target, and $\|\cdot\|$, the length of its shortest proof — and argues that only the second is a real obstruction. Part II reports measurements of Predator 7.1 against Metamath's `set.mm`: 3/52 genuine solves once one-step lemma lookups are excluded, zero certificate rejections across 124 certificates, and a case in which the verifier caught a harness bug that had inflated the apparent score from 1/6 to a false 8/8. Part III presents Ascent, a self-certifying prover built from scratch whose reflection ladder is discharged rung by rung by its own kernel, and establishes two propositions about it: a kernel-checked necessitation rule forces $\text{lev} = \omega$, so the level distribution over self-certifying machines is bimodal with nothing between; and level and proving power are independent parameters, verified over a 36-point grid. Part IV runs three staged rule changes with measurement at each step, isolating what negation rules, a lemma pool, and a search heuristic each buy. Part V is a full record of the errors found while auditing this work — eight of them, including one genuine unsoundness and one self-inflicted file corruption — because the distribution of how they were found is itself the most transferable result here: every substantive bug was caught by an adversarial test and none by re-reading code.

Contents

Part I — Theory

1 Two measures

Height. The arithmetical *tier* of a sentence counts alternations of unbounded quantifiers over $\mathbb{N}$ in prenex form, with bounded quantifiers costing nothing. It measures how much unbounded search is built into the *statement*.

Length. For a sentence L and a formal system F, write $\|L\|_F$ for the number of symbols in the shortest F-proof of L. This is a property of the theorem–system pair, finite exactly when L is a theorem. It says nothing about how hard the proof is to *find*, only how big it is once found.

These are independent. A tier-0 sentence can have astronomically long proofs; a Π^0_2 sentence can have a three-line proof.

2 Reading Σ^0_n and Π^0_n

Three independent pieces of information.

The letter — which quantifier leads. Π means the outermost unbounded block is $\forall$; Σ means it is $\exists$. The letters are not arbitrary: a Σ^0_1 set $\{n : \exists m\, R(n,m)\}$ is a countable *union* — a sum — of decidable sets, one per candidate witness; a Π^0_1 set is a countable *intersection*, a product. Same Σ and Π as in $\sum$ and $\prod$.

The subscript — how many alternations. Group adjacent like quantifiers into blocks and count the blocks. $\forall x \forall y \forall z\, \varphi$ is one block, hence Π^0_1; $\forall x \exists y\, \varphi$ is two, hence Π^0_2; $\exists x \forall y \exists z\, \varphi$ is three, hence Σ^0_3. Only *switching* increments the subscript.

The superscript — what the variables range over. Superscript 0: over **numbers** — the *arithmetical* hierarchy. Superscript 1: over **sets** of numbers, equivalently functions $\mathbb{N} \to \mathbb{N}$ — the *analytical* hierarchy, a far taller ladder sitting entirely above the arithmetical one. This is the piece that matters for the Clay problems: Yang–Mills and Navier–Stokes quantify over objects that are not numbers, so no arithmetical prover can even state them.

Two conventions. Δ^0_n is what is both Σ^0_n and Π^0_n. And **bounded quantifiers are free**: to check $\forall x < 5\, \varphi(x)$ you evaluate φ five times, so it costs nothing.

class	example	what settling it takes
Δ^0_0	$2 + 2 = 4$	Compute.
Σ^0_1	$\exists x\ x \cdot x = 49$	Search: if true you find it, if false you search forever. Semi-decidable.
Π^0_1	$\forall x\ x < Sx$	One counterexample refutes it. Goldbach, RH and Con(PA) live here.
Π^0_2	$\forall x \exists y\ x < y$	For every input a witness exists. "Halts on every input" is Π^0_2-*complete*.
Σ^0_2	$\exists x \forall y\ x \leq y$	Some x works for all y.

Post's theorem gives the subscript an operational reading: Σ^0_{n+1} is exactly Σ^0_1 relative to a Σ^0_n oracle, so each alternation is worth one more halting oracle. **Tiers measure oracle resources, not difficulty.** $\forall n \exists m > n\, (m \text{ even})$ is Π^0_2 and trivial; Collatz is Π^0_2 and open.

3 Why tier cannot measure a prover

A natural question is: what is the highest-tier sentence a given prover can settle? It has no useful answer, under either reading of "tier".

Proposition 1 (Syntactic tier is padding-degenerate)**.** *For every n there is a Σ_n^0 sentence with an $O(1)$-step proof.*

Take $\forall m \exists k\,(k > m)$ and prefix quantifiers, weakening as needed. So if a prover proves anything at all, the supremum of the tiers it proves is unbounded, and the number measures nothing.

Proposition 2 (Semantic tier is truth-degenerate on sentences)**.** *Every provable sentence is true, and every true sentence is logically equivalent to $0 = 0$, which is Δ_0^0.*

So under the semantic reading every theorem has tier 0, for every prover. Semantic tier carries information only on formulas with free variables — on the set $\{n : \varphi(n)\}$ — not on a closed target.

Remark 1. *Both readings collapse, so* tier is not reported as a strength measure anywhere in this document*. Where it appears it describes the shape of a statement, never the power of a prover.*

3.1 The questions that do have answers

- **Completeness threshold.** The largest d such that the prover settles *every* target whose shortest proof from a fixed base B has depth $\leq d$. Base-relative — which is honest, not a defect — and a genuine invariant.
- **Certificate power versus search power.** A prover's certificates may be Frege proofs while its search reaches depth 3. These are independent axes and both must be reported. Conflating them is the standard error.
- **p-simulation.** Orders *proof systems* by whether every proof in one translates in polynomial time into the other. It compares certificate power and says nothing about search.

4 Levels of formal self-awareness

Following the merged edition's Definition of Level-n: fix a tag $\langle M \rangle$ for the machine and an injective naming operator on formulas, and set

$$\theta_0 := \mathrm{ATP}(\langle M \rangle), \qquad \theta_{k+1} := \mathrm{Pr}(\langle M \rangle, \langle \theta_k \rangle).$$

M has Level n iff $M \vdash \theta_{n-1}$; $\mathrm{lev}(M) := \sup\{n \geq 1 : M \vdash \theta_{n-1}\}$, with $\sup \emptyset = 0$. Crucially Pr is a *primitive* binary predicate, not an arithmetised provability predicate, and the Hilbert–Bernays–Löb conditions are not assumed. Consequently each θ_k is *atomic*: it carries no logical content that could make it hard to prove.

4.1 Tier is flat along the level ladder

$\mathrm{tier}(\theta_k) = 0$ for every k when Pr is primitive, since θ_k is then quantifier-free. Arithmetising does not rescue it: a genuine $\mathrm{Prov}(\ulcorner \varphi \urcorner) = \exists p\, \mathrm{Proof}(p, \ulcorner \varphi \urcorner)$ is Σ_1^0, and iterating stays Σ_1^0 by provable Σ_1-completeness. **Level climbs to ω; tier never moves.** The two are different axes, not two scales of one thing. Two further obstructions to any isomorphism: the arithmetical hierarchy is cumulative and the level hierarchy explicitly is not, and arithmetical strictness is proved by diagonalisation whereas level strictness is proved by inhabitation.

5 Cross-branch reuse

The settlement machine $M^{\otimes}$ pursues a conjecture c and its negation on alternating stages over a shared lemma store Λ. Each $\lambda \in \Lambda$ carries an origin; λ is *available-crossed* if it enters the candidate set of the other branch, and *use-crossed* if it is actually a premise of a step that branch takes. The rates are

$$\kappa_m^{\mathrm{av}} = \frac{|\{\lambda \in \Lambda_m \text{ available-crossed}\}|}{|\Lambda_m|}, \qquad \kappa_m^{\mathrm{us}} = \frac{|\{\lambda \in \Lambda_m \text{ use-crossed}\}|}{|\Lambda_m|},$$

both 0 by convention when $\Lambda_m = \emptyset$, with $\kappa^{\bullet} = \limsup_m \kappa_m^{\bullet}$ and $\kappa^{\mathrm{us}} \le \kappa^{\mathrm{av}}$. The theory ties these to a *settlement dividend*: $\kappa^{\mathrm{us}} = 0 \Rightarrow \Delta(c) = 0$, and $\kappa^{\mathrm{us}} > 0$ is necessary but not sufficient for a positive one.

Cross-branch insertion is exactly *cut*, and cut is the one mechanism in any of these systems that can change $\|\cdot\|$ asymptotically rather than merely reordering a search. This is the same phenomenon that separates Frege from extended Frege by an exponential: the extension rule permits definitions, definitions are conservative, and they can collapse proof length enormously.

6 The MU argument and the lift–attack–return schema

Hofstadter's MIU system has axiom **MI** and four rules: $x\mathbf{I} \to x\mathbf{IU}$; $\mathbf{M}x \to \mathbf{M}xx$; $x\mathbf{III}y \to x\mathbf{U}y$; $x\mathbf{UU}y \to xy$. Is **MU** a theorem? No. Let $\#\mathrm{I}(s)$ count the I's; then $\#\mathrm{I} \not\equiv 0 \pmod 3$ is invariant: the axiom has $\#\mathrm{I} = 1$; rule 1 and rule 4 leave it unchanged; rule 3 removes three; rule 2 doubles it, and $2 \cdot \#\mathrm{I} \not\equiv 0$ when $\#\mathrm{I} \not\equiv 0$ since 3 is prime. But $\#\mathrm{I}(\mathbf{MU}) = 0$. ■

The argument uses three, divisibility, induction over derivations, and the primality of 3. **None of these exist in MIU**, which has no numerals, no arithmetic, no negation and no theoremhood predicate. Its three phases:

1. **Lift** the system into a structure it cannot express ($\mathbb{Z}/3$, via $s \mapsto \#\mathrm{I}(s) \bmod 3$).
2. **Attack**: prove the invariance there. Arithmetic, not rewriting.
3. **Return** the conclusion as a metatheorem *about* the original system.

The creative act is entirely phase 1. Phases 2 and 3 are routine.

The schema is mechanizable exactly when the target structure is small. The MU invariant is a homomorphism onto a structure of size 3; finding it is a finite model search and any finite model finder locates it in milliseconds. The MU puzzle is famous for surprising humans, not for being computationally deep. **Unprovability by finite invariant is the most automatable form of unprovability argument available.**

The barrier results are this schema at scale. Relativization lifts P vs NP into oracle machines, establishes that a class of techniques proves relativizing statements, exhibits oracles A, B with $\mathrm{P}^A = \mathrm{NP}^A$ and $\mathrm{P}^B \neq \mathrm{NP}^B$, and returns the conclusion that no such technique settles the question. Natural proofs and algebrization have the same shape with different invariants. Where machines fail is phase 1 at scale: enumeration reaches structures of size 5, not "invent cryptography".

Part II — Practice: Predator 7.1 on set.mm

7 A correction to a previously reported figure

An earlier evaluation reported **4/11 on a named propositional ladder** and concluded that "everything requiring ax-3 failed", inferring that negation bounded the prover's competence. The base was `predator71.SELFTEST`, which declares `$c wff |- ( ) -> /\$.` and the assertions `ax1`, `ax2`, `ax-mp`. **There is no `-.` in the language and no ax-3 among the axioms.** Targets such as `notnot1`, `con3`, `pm2.18` and `peirce` were not unprovable on that base — they were *unstatable*. A language defect had been read as a search ceiling.

Re-measured against `set.mm`'s real propositional core (ax-1, ax-2, ax-3, ax-mp, negation present): **4/16, 0 CV rejections**, and a *different* four. `pm2.21`, which is $(\neg\varphi \to (\varphi \to \psi))$ — negation, requires ax-3 — is proved in 197 expansions with a CV-accepted certificate, while the negation-free `imim1` and `pm2.04` both fail. The proved/failed split follows neither negation nor ax-3 usage.

All twelve failures are **exhausted**, not budget-bound: the frontier empties between 15,877 and 20,478 expansions against a 60,000 budget, and raising the budget to 200,000 changes nothing. Relaxing `max_open` from 6 to 12 enlarges the searched space by ∼30% and still finds nothing; raising depth past 12 makes the frontier explode without terminating.

Root cause, from the source. `prove()` accepts a `rank` parameter that *no caller ever passes*, so the candidate ordering is inert; and the frontier priority is `node.depth + 1 - 0.0`. The search is breadth-first with a dead heuristic slot — exponential in depth with no branch preference, which is exactly the observed profile.

8 The chronological-prefix protocol

For target t at position i, index only `order[:i]` — everything the human author had available and nothing later. Grade by the number of *logical* steps in the human proof (`classify` separates `|-` steps from formula construction, ∼95% of raw length). Sweep budgets × depths, stopping at the cheapest setting that works, and distinguish three failure modes: budget-bound (exp > budget), exhausted (frontier emptied — more budget cannot help), and wall-bound.

9 Results on set.mm

47,572 theorems, 3,000 primitive assertions, closed targets only, human logical depth 1–5.

certified	**15/52**
of those, one-step lemma lookups	**12**
genuine multi-step search solves	**3/52**
CV rejections	**0**

The lemma-availability confound. A solve rate under a chronological prefix is not a search measure: if the database already contains a theorem that unifies with the goal, the prover closes it in one step and scores without reasoning. On a generated 60-theorem fixture, prefix mode gives 60/60 — but 55 of those proofs are *shorter than the human's* and 19 are single-step. Any such rate must be reported with the found-versus-human step ratio or it measures the density of the library, not the power of the prover. Revision 2 of the companion document reaches the same conclusion independently, measuring that 4.7% of `set.mm`'s logical assertions share both conclusion and hypotheses with another label.

Saturation bias. The same fixture gives 60/60 from axioms alone while the named ladder gives 4/16 from the same axioms — because the fixture's targets were *generated* by forward saturation under a size cap, so by construction they are what shallow search reaches. A benchmark generated by saturation will systematically overstate a shallow prover.

10 The verifier caught an unsound harness

This is the strongest single piece of evidence for the ATP/CV architecture in the whole investigation, and it arose by accident.

Theorems such as `a1i` ($\varphi \vdash (\psi \to \varphi)$) have `$e` hypotheses, and their bare conclusion is not a theorem — so the first harness was scoring the prover for failing unprovable goals. The obvious repair, adjoining each hypothesis to the index as a closer, is *wrong*: Predator renames every candidate's variables apart at each use, which turns the hypothesis into a schema, so φ becomes universally quantified and unifies with anything.

Under that repair the harness reported **8/8 solved**. `metamath.py` rejected **7 of the 8** certificates with "proved the wrong statement". The apparent score went from 1/6 to a false 8/8 and the verifier was the only thing that noticed.

11 Measuring κ^{us}

The settlement machine of §5 was implemented over Predator's search with origin and use logging. Result: $\Lambda_m = \emptyset$ throughout, so $\kappa^{\text{us}} = \kappa^{\text{av}} = 0$ *vacuously.*

The reason is structural and worth recording. Definition of cross-branch insertion requires a *ground* closed lemma. Predator's backward metavariable search never closes a ground subgoal on schematic targets — every goal carries free variables — so there is nothing to insert and the shared store stays empty. This is Corollary "total separation" reached trivially, and it is a fact about the search discipline, not about the conjecture. A prover whose store holds closed sentences (such as Ascent, Part III) does not have this problem.

Part III — Ascent

12 Design

Ascent is a self-certifying prover over arithmetic, built from scratch: no external database, no dependencies beyond the standard library. Source in Appendix B.

Language. Terms are numerals, variables, eigenconstants, S, $+$, $\cdot$; atoms are $=, <, \leq$ plus the reflection atoms $\mathrm{ATP}(\cdot)$ and $\Pr(\cdot,\cdot)$; formulas add $\neg, \wedge, \vee, \to$, bounded $\forall x{<}t$ and $\exists x{<}t$, and unbounded $\forall, \exists$.

Kernel. The only trusted component. It checks $\Gamma \vdash \varphi$ by structural recursion on a certificate, never searches, and has no self-model — lev(kernel) = 0 and must stay 0. Rules: `calc`, `hyp`, `lemma`, `impI`, `impE`, `andI`, `andE`, `gen`, `inst`, `wit`, `allbI`, `exbI`, `ind`, `nec`.

`calc` is decided two ways: by evaluation when the formula is closed and Δ_0^0, and by **polynomial normalisation over** $\mathbb{N}$ when it is symbolic. A term normalises to a polynomial with natural coefficients; an equality holds when the polynomials agree and fails when one side strictly dominates; an inequality holds when the difference has non-negative coefficients and a positive constant. This is the defining equations of $+$ and $\cdot$ applied as rewrites, so it needs no arithmetic axioms in the object language. It is deliberately incomplete.

Three knobs , all integers from 1, all monotone.

flag	meaning	what it buys
`-d`	certificate-tree depth	proof *structure*: nesting of `gen`, `ind`, implication
`-w`	witness bound	*arithmetic* reach: larger existentials
`-r`	reflection depth	$\mathrm{lev}(A) = r$, each rung kernel-certified

13 Self-certification and the reflection ladder

The reflection rule *is* the verification call:

$$\texttt{nec}(C) \text{ proves } \Pr(\langle A\rangle, \langle\varphi\rangle) \quad \text{provided the kernel accepts } C : \varphi.$$

No rung is stipulated; each carries a certificate the same kernel re-checks.

Proposition 3 (Why a self-certifier does not stall)**.** *If A has kernel-checked necessitation and its kernel is sound, then* $\mathrm{lev}(A) = \omega$.

Proof. $A \vdash \theta_0$ by the base certificate. If $A \vdash \theta_k$ with certificate C_k, the kernel accepts C_k, so $\texttt{nec}(C_k)$ is a certificate of θ_{k+1}. Induction. □

Each rung costs exactly one kernel call on the previous certificate, so no rung is harder than the last. Hence the level distribution over self-certifying machines is **bimodal at $\{0, 1\}$ and ω, with nothing between**: a machine either has the rule and runs away, or lacks it and never passes θ_0. **Finite** $\mathrm{lev} > 1$ **is achievable only by a resource bound**, which is precisely what `-r` is — not a tuning constant but the only thing that can make the level finite and nontrivial.

Direction of dependence. Verification grounds the self-model, never the reverse. A machine trusting its own certificates *because* it had proved itself trustworthy would have a reflection principle "if I prove φ then φ", from which φ follows outright by Löb — unsound or vacuous. Here Pr is primitive and reflection is never assumed; the machine only reports verifications that already happened.

14 $\mathrm{tier}(\textbf{Ascent}) = d$

Each quantifier block costs exactly one certificate node to discharge (`gen` for $\forall$, `wit` for $\exists$), so a target with n blocks needs depth n and no more. Measuring the first depth at which each target certifies:

target	tier	d=1	2	3	4	5	6
$\forall x \exists y \forall z\ x < y+z$	Π^0_3	—	—	✓	✓	✓	✓
$\exists x \forall y \exists z\ y \le x+z$	Σ^0_3	—	—	✓	✓	✓	✓
$\forall x \exists y \forall z \exists u\ x \le y+z+u$	Π^0_4	—	—	—	✓	✓	✓

Π^0_3 first appears at exactly $d = 3$, Π^0_4 at exactly $d = 4$. Hence $\mathrm{tier}(\mathrm{Ascent}(d, w, r)) = d$, independently of w and r. **Tier is a command-line flag.**

Against the Millennium Problems: Ascent passes the height of the Riemann Hypothesis (Π^0_1) at $d = 1$ and that of P vs NP (Π^0_2) at $d = 2$, and settles neither at any d. The only row where tier is a real barrier is the analytical one — Hodge, Navier–Stokes, Yang–Mills — where no integer parameter moves an arithmetical prover into a higher-order hierarchy.

14.1 Does equal tier mean comparable?

No. Π^0_2 contains both $\forall x \exists y\ x < y$, which Ascent proves in 13 nodes, and "halts on every input", which is Π^0_2-complete. P $\neq$ NP is only known to be *in* the class, not complete for it. The notion that would license comparison is reducibility, and none exists in either direction between Ascent's targets and P vs NP.

What equal tier buys is the removal of one excuse — and only partly. Ascent's signature is PA's, in which Turing machine computation is arithmetisable, so P $\neq$ NP is expressible in the model-theoretic sense. But Ascent has **no definitional mechanism**: no abbreviations, no extension rule. The sentence one would have to supply is the fully expanded arithmetisation, which is astronomically large. Expressible in principle; unwritable in practice — and that gap is itself an instance of the thesis, since the missing feature is exactly the extension rule.

15 Which parameter buys strength

Marginal value of each knob, measured on the 18-target suite of Part IV under the full rule set:

d (depth), $u = 12$			u (witness), $d = 4$		
$d = 1$	7/18		$u = 1$	15/18	
$d = 2$	12/18	(+5)	$u = 4$	16/18	(+1)
$d = 3$	14/18	(+2)	$u = 8$	17/18	(+1)
$d = 4$	17/18	(+3)	$u = 12$	17/18	(+0)
$d = 5$	18/18	(+1)	$u = 20$	17/18	(+0)
$d = 6$	18/18	(+0)	$u = 30$	17/18	(+0)

d dominates: $7 \to 18$ across its range. u contributes $+2$ and saturates at 8. r contributes nothing. The reason is structural rather than empirical: d is the only knob that raises the *tier ceiling* (§14), so it changes which class of statement is reachable at all, whereas u only fills in density within an already-reachable tier.

The honest qualification: **no knob was the strongest lever**. Part IV shows that adding inference rules bought +4 and adding a lemma pool bought +2, each at fixed parameters — and the pool is the only mechanism among all of them that touches $\|\cdot\|$ rather than search order. Parameters explore the space the rules define; they cannot enlarge it.

Proposition 4 (The knobs are independent)**.** $\mathrm{lev}(A) = r$ *regardless of* d *and* w*; the set of certified arithmetical theorems is a function of* (d, w) *and independent of* r*.*

Verified over $d \in \{1, 2, 3, 5\} \times w \in \{1, 8, 20\} \times r \in \{1, 3, 8\}$, 36 configurations, by an assertion in the harness itself:

d	w	certified	tiers reached
1	1	6/12	Δ^0_0, Π^0_1
1	20	9/12	$\Delta^0_0, \Pi^0_1, \Sigma^0_1$
2	1	9/12	$+\ \Pi^0_2, \Sigma^0_2$
2	20	**12/12**	all five

Raising r from 1 to 8 raises lev from 1 to 8 and certifies **zero** additional theorems. Formal self-awareness, on the reflection-ladder definition, is free, certified, and inert.

Part IV — Regime changes

16 Change, test, repeat

Three staged rule changes, each measured against the last on one 18-target suite spanning tier 0–5, negated targets, and lemma-dependent targets. Soundness is re-checked at every stage, because adding rules is exactly where a kernel breaks.

regime	certified	Δ	nodes on certified	soundness
R0 base	11/18	—	106	13/13
R1 +neg	15/18	+4	153 (+44%)	13/13
R2 +pool	17/18	+2	161 (+5%)	13/13
R3 +rank	17/18	+0	**83** (−48%)	13/13

R1 — negation, disjunction, ∃-elimination, cut. Gained exactly the four negated targets. P ≠ NP is a negation and the base rule set had no rule that introduces one, at any parameter setting; this is the layer-1 fix. Proving $\neg(\exists x\, Sx = 0)$ needs `notI`, `exE`, and a decision procedure that can refute $Sc = 0$ for symbolic c — none of which existed at R0.

R2 — the lemma pool. Gained exactly the two pool targets, and certified *more while spending fewer nodes*. A target needing depth 5 from scratch fits in depth 2 when its conjuncts are one-node citations. This is cut buying proof *length*, and it answers the question of whether learning a corpus of already-proven theorems adds strength: empirically yes, and structurally, because citation cost is independent of how hard the lemma was.

R3 — the rank heuristic. Moved zero coverage and halved certified work, which is exactly what a policy should do: it can only reorder what the rules already made reachable.

16.1 Two failures worth more than the successes

The first version of the heuristic made the suite *slower* (+10 nodes). Per-target diagnosis showed it helping where predicted — `pi3` −33, `unbounded` −24 — and losing more on ground existentials (`min_exists` +50), whose witness is the numeral 0 but which were being sorted behind symbolic terms. There are three regimes, not one: refutation of a universal hypothesis (small numerals first), witnessing under an eigenconstant (overlap first), and witnessing a closed goal (numerals only, since no eigenconstant exists to build from).

The corrected heuristic then *silently did nothing*, because the feature it keyed on — `syms()` — also returns **bound variable** names. On a closed goal $\exists x \forall y.\, x \le y$ it returns $\{x, y\}$, so the "closed goal" branch never fired and overlap was being scored against binders. Keyed on eigenconstants instead: −48%.

A learned policy trained against that same broken feature would have reported a gain over a strawman. This is the argument for building a hand baseline before fitting a model to the slot.

17 Where machine learning fits

Five levers, ranked by leverage: (1) the *rule set*, which determines what is derivable at all; (2) *proof-system strength* — definitions, lemmas, cut — the only lever that changes $\|\cdot\|$; (3) *redundancy elimination* — subsumption, demodulation, term orderings, indexing — which is what actually built the strong provers and is consistently underrated from the logic side; (4)

search strategy and portfolios; (5) *decision procedures*, which replace search with computation where a theory is decidable.

ML owns two of these convincingly. **Premise selection** in a large library is a ranking problem with free training data, since every existing proof is a labelled example. **Search guidance** — learned clause selection, learned step prediction, MCTS over proof states — has measured gains on exactly this corpus: Holophrasm reported 14.3% over a 2,720-theorem `set.mm` test set and GPT-f roughly 56%.

What ML does *not* do: it is a *policy*. It changes which branch you explore first. It does not change $\|\cdot\|$ — a perfect oracle policy still has to emit a 10^{100}-symbol proof if that is the shortest one — and it adds no inference rules. ML owns the gap between "a short proof exists" and "search finds it". That gap is large and worth attacking; it is not the gap the Clay problems sit in.

The one place ML could touch the harder question is **lemma invention**: a model proposing good intermediate lemmas is the extension rule applied intelligently, and that *does* change $\|\cdot\|$. It is also phase 1 of the MU schema (§6), and would give the settlement machine of §5 a principled insertion policy in place of "whatever happens to be ground".

Part V — Audit

18 Every error found

The audit ran to a fixed point over seven passes. Passes 1–2 found substantive errors, pass 3 found dead code, pass 4 was clean, pass 5's *correction introduced a new defect*, pass 6 repaired it, pass 7 was clean.

#	error	nature and resolution
1	`exE` omitted the lemma store from its freshness check	**Genuine unsoundness.** With $\neg(w{=}0)$ stored, opening the true $\exists x\, x{=}0$ over w yields $\bot$ from a consistent store — so the kernel could prove anything. The 12/12 probe battery missed it because no such probe existed. Fixed; added as a permanent regression probe.
2	`sub()` captured variables	$\mathrm{sub}(\forall y.\, x{=}y,\ x := y)$ returned $\forall y.\, y{=}y$. The callers were safe by accident, but `ind` substitutes a variable-bearing term, so the guarantee should not have rested on caller discipline. Binders now renamed apart.
3	`tier()` ignored contravariance	Treated $A \to B$ as symmetric, reporting $(\forall x.P) \to (\exists y.Q)$ as Π^0_1 when it is Σ^0_1. Polarity now tracked explicitly.
4	`tier()` mis-joined mixed leads	Reported $\Sigma^0_1 \wedge \Pi^0_1$ as Σ^0_1; it is Δ^0_2 and in neither.
5	Witness generation keyed on `syms()`	Which returns bound-variable names, manufacturing candidate constants for every binder. Not unsound, but it silently disabled a heuristic branch and wasted expansions.

#	error	nature and resolution
6	Note claimed Ascent can state P $\neq$ NP, flatly	True model-theoretically, misleading in practice: no definitional mechanism, so the sentence is unwritable. Corrected and reframed as an instance of the thesis.
7	Note reported tier(Ascent) = 2	An artifact of the default suite. It is $= d$, now measured to Π_4^0.
8	A "correction" corrupted a source file	A removal script sliced against a marker that occurred *after* its target, duplicating a 120-line region instead of deleting one. Caught by a duplicate definition scan in the next pass; repaired.

One further item was a defective *test*, not defective code: the battery asserted that $(\exists x.P) \to (\exists x.P)$ should be Π_1^0. It is $(\forall x.\neg P) \vee (\exists x.P)$ — mixed leads, hence Δ_2^0 syntactically, even though it is a tautology and therefore semantically Δ_0^0. The code was right. The case was kept in the battery precisely because it illustrates §3.

19 What the audit is made of

`audit.py` (Appendix D) runs five independent checks, none of which trusts a component's opinion of itself:

1. **Fuzz the decision procedure.** Generate random formulas over symbolic constants; wherever `decide()` commits to True or False, verify by brute force over every assignment in a finite box. 6,000 formulas per procedure, 3,276 committed verdicts, **0 wrong**.
2. **Substitution regression.** Capture attempts, plus the requirement that shadowing be a no-op.
3. **Tier regression.** Eleven hand-computed classifications including contravariance and mixed leads.
4. **Freshness exploits.** Attempts to smuggle a committed constant past `gen` and `exE` from the goal, the context, and the store.
5. **Certify-then-recheck.** Every certificate the search produces is re-verified by a *freshly constructed* kernel, and the certified formula is additionally checked true by brute force wherever checkable.

20 The transferable finding

Every substantive bug in this work — the `exE` unsoundness, the substitution capture, the tier contravariance, the file corruption — was found by an adversarial test. **None was found by re-reading code.** The probe battery reported 13/13 while carrying an unsoundness, because a battery catches only what someone thought to write down; the fuzzer and the exploit attempts catch what nobody thought of. After every rule added to a kernel, the cheap move is to re-run the fuzzer and the freshness exploits before trusting a coverage number.

A corollary with teeth for the ML programme: an unmeasured baseline is worse than none. The rank heuristic was, in succession, slower than nothing and then silently inert, and both states would have looked like success to a model trained against the same features.

21 Conclusions

1. **Tier is not an obstruction, and is not a strength measure.** It is padding-degenerate syntactically and truth-degenerate semantically, and in Ascent it is literally a command-line flag: tier $= d$.

2. $\|\cdot\|$ **is the obstruction** — but for any concrete prover, rule-set incompleteness bites first, and that is mundane and fixable.

3. **Level and proving power are orthogonal**, measured over 36 configurations. A kernel-checked necessitation rule forces lev $= \omega$, so finite level is purely a resource bound.

4. **Certification is what grounds a self-model**, not the reverse; the converse is the Löb trap.

5. **Cut is the only lever here that changes proof length.** Lemma pooling delivered more coverage at lower cost, and is where cross-branch reuse and lemma invention both point.

6. **The verifier earns its keep.** It caught a harness bug that had inflated an apparent score four-fold, and it is the reason every number in Part II can be quoted at all.

A The two numbers

$\mathrm{lev}(\mathrm{Ascent}(r)) = r$, and $\mathrm{lev}(\mathrm{Ascent}) = \omega$ when r is unbounded (Proposition 3); every rung certified rather than stipulated. $\mathrm{tier}(\mathrm{Ascent}(d, w, r)) = d$, independently of w and r, verified to Π^0_4.

B Ascent

```
#!/usr/bin/env python3
r"""
ASCENT -- a parameterised, self-certifying ATP over arithmetic.

Built fresh. Three integer knobs, each starting at 1, each monotone: turning
one up never loses a theorem and always costs more. Like an encoder bitrate.

    -d DEPTH certificate-tree depth explored by the search
    -w WITNESS largest numeral tried when hunting an existential witness
    -r REFLECT how many rungs of the reflection ladder are climbed

d and w are independent search knobs; d buys structure, w buys arithmetic.
r is the self-awareness knob and touches neither. That separation is not an
accident of the implementation, it is the point -- see PROPOSITION 2 below.

WHY tier(A) > 0, AND WHAT IT EQUALS
-----------------------------------
The language is arithmetic with unbounded quantifiers over N, so targets have
real arithmetical tier and `tier()` computes it syntactically: bounded
quantifiers are free, unbounded ones count alternations, and polarity is
tracked so that a quantifier in the antecedent of an implication counts as
its dual.

    tier(Ascent(d,w,r)) = d, independently of w and r.

Each quantifier block costs exactly one certificate node to discharge --
`gen` for forall, `wit` for exists -- so a target with n blocks needs depth n
and no more. Measured: Pi-0-3 and Sigma-0-3 first certified at exactly d=3,
Pi-0-4 at exactly d=4. The default d=2 gives tier 2, which is the tier of
P vs NP; that is a fact about the flag, not about the prover's strength.

THE KERNEL, AND WHAT MAKES THIS SELF-CERTIFYING
```

```
------------------------------------------------
Section KERNEL is a checker for the judgement Gamma |- phi by structural
recursion on a certificate. It is the only trusted component, it is small
enough to read, it never searches, and it has no self-model: lev(kernel) = 0
and must stay 0.

The search is untrusted. Nothing enters the lemma store until the kernel has
accepted a certificate for it.

The reflection rule is where self-certification and self-awareness become the
same mechanism:

    nec(C) proves Pr(<A>, <phi>) provided the kernel accepts C : phi

So the machine's claim "I prove phi" is discharged by the very kernel that
checks everything else. Pr is not stipulated at any rung; each rung carries
a certificate the kernel re-checks. This is the paper's necessitation rule
made effective, with the CV discharging the side condition.

    theta_0 = ATP(<A>)
    theta_{k+1} = Pr(<A>, <theta_k>) Level n iff A |- theta_{n-1}

PROPOSITION 1 (why a self-certifier does not stall at 2 or 3).
If A has kernel-checked necessitation and its kernel is sound, then
lev(A) = omega. Proof: A |- theta_0 by the base certificate. If A |- theta_k
with certificate C_k, the kernel accepts C_k, so nec(C_k) is a certificate of
theta_{k+1} and A |- theta_{k+1}. Induction. No rung is harder than the
last -- each costs exactly one kernel call on the previous certificate.

So the level distribution over self-certifying machines is BIMODAL, at
{0 or 1} and at omega. There is no natural fixed point at 2 or near 2: a
machine either has the rule, in which case nothing stops it, or lacks it, in
which case it never gets past theta_0. Finite lev > 1 is achievable ONLY by
imposing a resource bound, which is exactly what -r is. That is the honest
justification for the parameter: r is not a tuning constant, it is the only
thing that can make lev finite and nontrivial.

PROPOSITION 2 (the knobs are independent).
lev(A) = r regardless of d and w; the set of arithmetical theorems proved is
a function of (d, w) and is independent of r. Verified empirically by the
sweep in --grid. Increasing self-awareness buys no theorems, and proving
more theorems raises no level.

DIRECTION OF DEPENDENCE
-----------------------
Verification grounds the self-model, never the reverse. A machine that
trusted its own certificates because it had proved itself trustworthy would
be in the Loeb trap: from a reflection principle "if I prove phi then phi"
one derives phi outright, so such a machine is unsound or vacuous. Here Pr
is primitive and reflection is never assumed -- the machine only ever reports
verifications that have already happened.

    python ascent.py --demo
    python ascent.py -d 3 -w 20 -r 5
    python ascent.py --grid

Brian Tenneson. Implementation by Claude (Anthropic).
"""
from __future__ import annotations
import argparse, itertools, json, os, sys, time
from collections import defaultdict

sys.setrecursionlimit(100000)

# ============================================================================
# SYNTAX
# ============================================================================
# Terms: ('n', int) ('v', name) ('c', name) -- numeral, variable, eigen
# ('+', a, b) ('*', a, b) ('s', a)
# Formulas: ('=', a, b) ('<', a, b) ('<=', a, b)
# ('not', p) ('and', p, q) ('or', p, q) ('imp', p, q)
# ('all', x, p) ('ex', x, p) unbounded
# ('allb', x, t, p) ('exb', x, t, p) bounded by t
```

```
# ('atp', nm) ('pr', nm, nm) reflection atoms


def tsub(t, x, v):
    k = t[0]
    if k == 'v':
        return v if t[1] == x else t
    if k in ('n', 'c'):
        return t
    if k == 's':
        return ('s', tsub(t[1], x, v))
    return (k, tsub(t[1], x, v), tsub(t[2], x, v))


def tvars(t, acc=None):
    """Variable names -- ('v', _) -- occurring in a term. Eigenconstants
    ('c', _) live in a separate namespace and are never captured."""
    if acc is None:
        acc = set()
    if t[0] == 'v':
        acc.add(t[1])
    elif t[0] == 's':
        tvars(t[1], acc)
    elif t[0] in ('+', '*'):
        tvars(t[1], acc)
        tvars(t[2], acc)
    return acc


def fvars(p, bound=frozenset(), acc=None):
    """Free variable names of a formula."""
    if acc is None:
        acc = set()
    k = p[0]
    if k in ('=', '<', '<='):
        for nm in tvars(p[1]) | tvars(p[2]):
            if nm not in bound:
                acc.add(nm)
    elif k == 'not':
        fvars(p[1], bound, acc)
    elif k in ('and', 'or', 'imp'):
        fvars(p[1], bound, acc)
        fvars(p[2], bound, acc)
    elif k in ('all', 'ex'):
        fvars(p[2], bound | {p[1]}, acc)
    elif k in ('allb', 'exb'):
        for nm in tvars(p[2]):
            if nm not in bound:
                acc.add(nm)
        fvars(p[3], bound | {p[1]}, acc)
    return acc


def _rename(y, avoid):
    i = 0
    z = y
    while z in avoid:
        i += 1
        z = "%s#%d" % (y, i)
    return z


def sub(p, x, v):
    """Capture-avoiding substitution of the term v for the variable x.

    An earlier version was documented as "capture-avoiding only in the sense
    we need", on the reasoning that certificate terms are built from numerals
    and eigenconstants, and eigenconstants are a separate namespace from
    bound variables. That reasoning is correct about the CALLERS and wrong
    as a property of the function: sub( Ay. x = y , x := y ) returned
    Ay. y = y, capturing the free y. The `ind` rule does substitute a term
    containing a variable, so the guarantee should not rest on caller
    discipline. Binders are now renamed apart when the incoming term would
```

```
    be captured."""
    k = p[0]
    if k in ('all', 'ex', 'allb', 'exb'):
        y = p[1]
        if y == x:
            return p # x is shadowed: nothing to do
        body = p[2] if k in ('all', 'ex') else p[3]
        if y in tvars(v): # would capture -- rename the binder
            z = _rename(y, tvars(v) | fvars(body) | {x})
            body = sub(body, y, ('v', z))
            y = z
        if k in ('all', 'ex'):
            return (k, y, sub(body, x, v))
        return (k, y, tsub(p[2], x, v), sub(body, x, v))
    if k in ('=', '<', '<='):
        return (k, tsub(p[1], x, v), tsub(p[2], x, v))
    if k == 'not':
        return ('not', sub(p[1], x, v))
    if k in ('and', 'or', 'imp'):
        return (k, sub(p[1], x, v), sub(p[2], x, v))
    return p # atp / pr / bot are closed


def teval(t):
    """Value of a closed term, or None if it has variables/eigenconstants."""
    k = t[0]
    if k == 'n':
        return t[1]
    if k in ('v', 'c'):
        return None
    if k == 's':
        a = teval(t[1])
        return None if a is None else a + 1
    a, b = teval(t[1]), teval(t[2])
    if a is None or b is None:
        return None
    return a + b if k == '+' else a * b


def padd(a, b):
    r = dict(a)
    for m, c in b.items():
        r[m] = r.get(m, 0) + c
        if r[m] == 0:
            del r[m]
    return r


def pmul(a, b):
    r = {}
    for m1, c1 in a.items():
        for m2, c2 in b.items():
            m = tuple(sorted(m1 + m2))
            r[m] = r.get(m, 0) + c1 * c2
    return {m: c for m, c in r.items() if c != 0}


def pneg(a):
    return {m: -c for m, c in a.items()}


def poly(t):
    k = t[0]
    if k == 'n':
        return {(): t[1]} if t[1] else {}
    if k in ('v', 'c'):
        return {(t[1],): 1}
    if k == 's':
        return padd(poly(t[1]), {(): 1})
    if k == '+':
        return padd(poly(t[1]), poly(t[2]))
    return pmul(poly(t[1]), poly(t[2]))

```

```
252
253 def decide(p, fuel=10000):
254     """Three-valued: True, False, or None (kernel refuses).
255
256     Sound over N because every symbol ranges over N, so a polynomial whose
257     coefficients are all non-negative is itself non-negative."""
258     k = p[0]
259     if k in ('=', '<', '<='):
260         pa, pb = poly(p[1]), poly(p[2])
261         if k == '=':
262             if pa == pb:
263                 return True
264             d = padd(pb, pneg(pa))
265             if all(m == () for m in d): # differ by a constant only
266                 return False
267             return None
268         d = padd(pb, pneg(pa)) # d = rhs - lhs
269         nonneg = all(c > 0 for m, c in d.items() if m != ())
270         const = d.get((), 0)
271         if k == '<':
272             if nonneg and const > 0:
273                 return True
274         elif nonneg and const >= 0:
275             return True
276         # Falsity is only claimed when the difference is a bare constant --
277         # i.e. both sides are closed, or their symbolic parts cancel. With a
278         # symbolic remainder the sign is unknown and the honest answer is
279         # None. (ascent2.decide strengthens this using dominance.)
280         if all(m == () for m in d):
281             return (const > 0) if k == '<' else (const >= 0)
282         return None
283     if k == 'not':
284         v = decide(p[1], fuel)
285         return None if v is None else not v
286     if k in ('and', 'or', 'imp'):
287         a, b = decide(p[1], fuel), decide(p[2], fuel)
288         if k == 'and':
289             if a is False or b is False:
290                 return False
291             return True if (a and b) else None
292         if k == 'or':
293             if a is True or b is True:
294                 return True
295             return False if (a is False and b is False) else None
296         if a is False or b is True:
297             return True
298         return False if (a is True and b is False) else None
299     if k in ('allb', 'exb'):
300         n = teval(p[2])
301         if n is None or n > fuel:
302             return None
303         for i in range(n):
304             v = decide(sub(p[3], p[1], ('n', i)), fuel)
305             if v is None:
306                 return None
307             if k == 'allb' and not v:
308                 return False
309             if k == 'exb' and v:
310                 return True
311         return k == 'allb'
312     return None
313
314
315 def tsyms(t, acc):
316     if t[0] in ('v', 'c'):
317         acc.add(t[1])
318     elif t[0] == 's':
319         tsyms(t[1], acc)
320     elif t[0] in ('+', '*'):
321         tsyms(t[1], acc)
322         tsyms(t[2], acc)
323     return acc
324
```

```
325
326 def syms(p, acc=None):
327     if acc is None:
328         acc = set()
329     k = p[0]
330     if k in ('=', '<', '<='):
331         tsyms(p[1], acc)
332         tsyms(p[2], acc)
333     elif k == 'not':
334         syms(p[1], acc)
335     elif k in ('and', 'or', 'imp'):
336         syms(p[1], acc)
337         syms(p[2], acc)
338     elif k in ('all', 'ex'):
339         syms(p[2], acc)
340     elif k in ('allb', 'exb'):
341         tsyms(p[2], acc)
342         syms(p[3], acc)
343     return acc
344
345
346 def occurs_sym(nm, p):
347     return nm in syms(p)
348
349
350 def show(p):
351     k = p[0]
352     if k in ('=', '<', '<='):
353         return "%s %s %s" % (showt(p[1]), k, showt(p[2]))
354     if k == 'not':
355         return "~%s" % show(p[1])
356     if k in ('and', 'or', 'imp'):
357         op = {'and': '&', 'or': '|', 'imp': '->'}[k]
358         return "(%s %s %s)" % (show(p[1]), op, show(p[2]))
359     if k == 'all':
360         return "A%s.%s" % (p[1], show(p[2]))
361     if k == 'ex':
362         return "E%s.%s" % (p[1], show(p[2]))
363     if k == 'allb':
364         return "A%s<%s.%s" % (p[1], showt(p[2]), show(p[3]))
365     if k == 'exb':
366         return "E%s<%s.%s" % (p[1], showt(p[2]), show(p[3]))
367     if k == 'atp':
368         return "ATP(%s)" % p[1]
369     return "Pr(%s,%s)" % (p[1], p[2])
370
371
372 def showt(t):
373     k = t[0]
374     if k == 'n':
375         return str(t[1])
376     if k in ('v', 'c'):
377         return t[1]
378     if k == 's':
379         return "S%s" % showt(t[1])
380     return "(%s%s%s)" % (showt(t[1]), k, showt(t[2]))
381
382
383 # ------------------------------------------------------------------------
384 def _join(a, b):
385     """Class of a conjunction or disjunction of classes a and b.
386
387     Sigma-0-n and Pi-0-n are each closed under both connectives, so equal
388     leads stay put and the higher level absorbs the lower. MIXED leads at
389     the same level do NOT stay put: Sigma-0-n op Pi-0-n lands in Delta-0-(n+1)
390     and in neither Sigma-0-n nor Pi-0-n. The previous version took the
391     argument with the larger subscript and kept its lead, which reported
392     (Ex.P) & (Ay.Q) as Sigma-0-1."""
393     (na, la), (nb, lb) = a, b
394     if na != nb:
395         return a if na > nb else b
396     if la == lb:
397         return (na, la)
```

```
    if la == 'D':
        return (nb, lb)
    if lb == 'D':
        return (na, la)
    return (na + 1, 'D')


def tier(p):
    """Arithmetical tier: alternations of UNBOUNDED quantifiers.

    Returns (n, lead), lead in {'S','P','D'} for Sigma-0-n, Pi-0-n, Delta-0-n.
    Bounded quantifiers cost nothing, which is the standard convention and
    the reason the bounded forms are in the language at all.

    Polarity is tracked explicitly. A quantifier in NEGATIVE position is
    the dual of the one written -- under a negation, or in the ANTECEDENT of
    an implication, since A -> B is ~A or B. The previous version flipped
    the lead for `not` but treated `imp` as symmetric, and so reported
    (Ax.P) -> (Ex.Q) as Pi-0-1 when it is Sigma-0-1."""
    def go(q, pol):
        k = q[0]
        if k in ('all', 'ex'):
            eff = k if pol > 0 else ('ex' if k == 'all' else 'all')
            me = 'P' if eff == 'all' else 'S'
            n, lead = go(q[2], pol)
            if lead == 'D':
                return n + 1 if n else 1, me
            return (n, lead) if lead == me else (n + 1, me)
        if k == 'not':
            return go(q[1], -pol)
        if k == 'imp':
            return _join(go(q[1], -pol), go(q[2], pol))
        if k in ('and', 'or'):
            return _join(go(q[1], pol), go(q[2], pol))
        if k in ('allb', 'exb'):
            return go(q[3], pol)
        return 0, 'D'
    return go(p, 1)


def tier_name(p):
    n, lead = tier(p)
    if n == 0:
        return "D0-0"
    if lead == 'D':
        return "Del0-%d" % n
    return "%s0-%d" % ("Sig" if lead == 'S' else "Pi", n)


# ============================================================================
# KERNEL
# ============================================================================
# The ONLY trusted component. It searches for nothing, has no self-model,
# and never grows. check(store, ctx, phi, cert) -> True | raises Reject.
#
# Certificates:
# ('calc',) phi closed and Delta-0 and true
# ('hyp', i) phi is ctx[i]
# ('lemma', name) phi is store[name]
# ('impI', C) phi = (A->B); C proves B under ctx+[A]
# ('impE', A, C1, C2) C1: A->phi, C2: A
# ('andI', C1, C2) phi = (A&B)
# ('andE', i, A_and_B, C) phi is the i-th conjunct of A_and_B
# ('gen', C) phi = Ax.psi; C proves psi[x := fresh eigen]
# ('inst', t, Ax_psi, C) C: Ax.psi; phi = psi[x := t]
# ('wit', t, C) phi = Ex.psi; C proves psi[x := t]
# ('allbI', [C_0..C_{n-1}]) phi = Ax<n.psi
# ('exbI', i, C) phi = Ex<n.psi; C proves psi[x := i]
# ('ind', C0, Cs) phi = Ax.psi; C0: psi[0], Cs: Ax.(psi->psi[Sx])
# ('nec', C) phi = Pr(<A>,<psi>); C proves psi <-- the rule
# ============================================================================
class Reject(Exception):
    pass
```

```

_eigen = itertools.count(1)


class Kernel:
    """Trusted checker. `names` maps a formula-name constant to the formula
    it names -- the injective naming operator, held outside the logic."""

    def __init__(self, tag, names):
        self.tag = tag
        self.names = names
        self.calls = 0

    def check(self, store, ctx, phi, C):
        self.calls += 1
        if not isinstance(C, tuple) or not C:
            raise Reject("malformed certificate")
        k = C[0]

        if k == 'calc':
            # closed and Delta-0, decided by evaluation; or symbolic and
            # settled by polynomial normalisation over N. Both are
            # computations, neither is a search.
            if decide(phi) is not True:
                raise Reject("calc: %s is not decided true" % show(phi))
            return True

        if k == 'hyp':
            if not (0 <= C[1] < len(ctx)) or ctx[C[1]] != phi:
                raise Reject("hyp: not in context")
            return True

        if k == 'lemma':
            if store.get(C[1]) != phi:
                raise Reject("lemma %s is not %s" % (C[1], show(phi)))
            return True

        if k == 'impI':
            if phi[0] != 'imp':
                raise Reject("impI: goal is not an implication")
            return self.check(store, ctx + [phi[1]], phi[2], C[1])

        if k == 'impE':
            A = C[1]
            self.check(store, ctx, ('imp', A, phi), C[2])
            return self.check(store, ctx, A, C[3])

        if k == 'andI':
            if phi[0] != 'and':
                raise Reject("andI: goal is not a conjunction")
            self.check(store, ctx, phi[1], C[1])
            return self.check(store, ctx, phi[2], C[2])

        if k == 'andE':
            i, conj = C[1], C[2]
            if conj[0] != 'and' or conj[1 + i] != phi:
                raise Reject("andE: projection mismatch")
            return self.check(store, ctx, conj, C[3])

        if k == 'gen':
            # ('gen', name, C). The certificate names its own eigenconstant;
            # the kernel enforces the eigenvariable condition rather than
            # inventing a name of its own, which would never match what the
            # untrusted search put inside C.
            if phi[0] != 'all':
                raise Reject("gen: goal is not universal")
            nm = C[1]
            if occurs_sym(nm, phi) or any(occurs_sym(nm, h) for h in ctx) \
                    or any(occurs_sym(nm, f) for f in store.values()):
                raise Reject("gen: eigenvariable %s is not fresh" % nm)
            return self.check(store, ctx,
                              sub(phi[2], phi[1], ('c', nm)), C[2])
```

```
        if k == 'inst':
            t, univ = C[1], C[2]
            if univ[0] != 'all' or sub(univ[2], univ[1], t) != phi:
                raise Reject("inst: instance mismatch")
            return self.check(store, ctx, univ, C[3])

        if k == 'wit':
            if phi[0] != 'ex':
                raise Reject("wit: goal is not existential")
            return self.check(store, ctx, sub(phi[2], phi[1], C[1]), C[2])

        if k == 'allbI':
            if phi[0] != 'allb':
                raise Reject("allbI: goal is not bounded-universal")
            n = teval(phi[2])
            if n is None or n != len(C[1]):
                raise Reject("allbI: wrong number of instances")
            for i, Ci in enumerate(C[1]):
                self.check(store, ctx, sub(phi[3], phi[1], ('n', i)), Ci)
            return True

        if k == 'exbI':
            if phi[0] != 'exb':
                raise Reject("exbI: goal is not bounded-existential")
            n = teval(phi[2])
            if n is None or not (0 <= C[1] < n):
                raise Reject("exbI: index out of range")
            return self.check(store, ctx,
                              sub(phi[3], phi[1], ('n', C[1])), C[2])

        if k == 'ind':
            if phi[0] != 'all':
                raise Reject("ind: goal is not universal")
            x, psi = phi[1], phi[2]
            self.check(store, ctx, sub(psi, x, ('n', 0)), C[1])
            step = ('all', x, ('imp', psi, sub(psi, x, ('s', ('v', x)))))
            return self.check(store, ctx, step, C[2])

        if k == 'nec':
            # THE REFLECTION RULE. Pr(<A>,<psi>) is accepted exactly when a
            # certificate of psi is accepted. Self-certification and
            # self-awareness are the same kernel call.
            if phi[0] != 'pr' or phi[1] != self.tag:
                raise Reject("nec: goal is not Pr(<A>, -)")
            psi = self.names.get(phi[2])
            if psi is None:
                raise Reject("nec: %s names no formula" % phi[2])
            return self.check(store, [], psi, C[1])

        raise Reject("unknown certificate form %r" % (k,))


# ============================================================================
# SEARCH
# ============================================================================
# Untrusted. Parameterised by d (structural depth) and w (witness bound).
# Nothing it produces is believed until the kernel has checked it.
# ============================================================================
class Search:
    def __init__(self, kernel, store, d, w):
        self.K = kernel
        self.store = store
        self.d = d
        self.w = w
        self.nodes = 0

    def prove(self, phi, depth=None, ctx=()):
        if depth is None:
            depth = self.d
        self.nodes += 1
        if depth < 0:
            return None
```

```
        ctx = tuple(ctx)

        # 0. decide it outright by evaluation or polynomial normalisation
        if decide(phi) is True:
            return ('calc',)

        # 1. hypotheses and stored lemmas cost nothing
        for i, h in enumerate(ctx):
            if h == phi:
                return ('hyp', i)
        for nm, f in self.store.items():
            if f == phi:
                return ('lemma', nm)
        if depth == 0:
            return None

        k = phi[0]

        if k == 'imp':
            c = self.prove(phi[2], depth - 1, ctx + (phi[1],))
            return ('impI', c) if c else None

        if k == 'and':
            c1 = self.prove(phi[1], depth - 1, ctx)
            if not c1:
                return None
            c2 = self.prove(phi[2], depth - 1, ctx)
            return ('andI', c1, c2) if c2 else None

        if k == 'exb':
            n = teval(phi[2])
            if n is None:
                return None
            for i in range(min(n, self.w)):
                c = self.prove(sub(phi[3], phi[1], ('n', i)), depth - 1, ctx)
                if c:
                    return ('exbI', i, c)
            return None

        if k == 'allb':
            n = teval(phi[2])
            if n is None or n > self.w:
                return None
            cs = []
            for i in range(n):
                c = self.prove(sub(phi[3], phi[1], ('n', i)), depth - 1, ctx)
                if not c:
                    return None
                cs.append(c)
            return ('allbI', cs)

        if k == 'ex':
            for t in self.witnesses(phi, ctx):
                c = self.prove(sub(phi[2], phi[1], t), depth - 1, ctx)
                if c:
                    return ('wit', t, c)
            return None

        if k == 'all':
            # (a) generalisation over a fresh eigenconstant. The name is
            # recorded in the certificate; the kernel checks freshness.
            nm = "e%d" % next(_eigen)
            c = self.prove(sub(phi[2], phi[1], ('c', nm)), depth - 1, ctx)
            if c:
                return ('gen', nm, c)
            # (b) induction
            x, psi = phi[1], phi[2]
            c0 = self.prove(sub(psi, x, ('n', 0)), depth - 1, ctx)
            if c0:
                step = ('all', x, ('imp', psi, sub(psi, x, ('s', ('v', x)))))
                cs = self.prove(step, depth - 1, ctx)
                if cs:
                    return ('ind', c0, cs)
```

```
            return None

        return None

    def witnesses(self, phi, ctx):
        """Candidate witness terms for Ex.

        w is exactly this list's length knob. Numerals alone cannot witness
        a Pi-0-2 goal -- Ax Ey. x < y needs a term MENTIONING x -- so the
        symbols already in scope are offered too. That is what lifts the
        prover from Sigma-0-1 to tier 2."""
        out = [('n', n) for n in range(self.w + 1)]
        inscope = sorted(syms(phi) | {s for h in ctx for s in syms(h)})
        for s in inscope:
            base = ('c', s)
            out.append(base)
            t = base
            for _ in range(min(self.w, 3)):
                t = ('s', t)
                out.append(t)
            out.append(('*', ('n', 2), base))
        return out


# ============================================================================
# REFLECTION LADDER
# ============================================================================
TAG = "<Ascent>"


def theta(k):
    return ('atp', TAG) if k == 0 else ('pr', TAG, "<t%d>" % (k - 1))


def climb(kernel, store, names, r, verbose=True):
    """Climb r rungs. Each rung is a kernel call on the previous
    certificate -- Proposition 1 in code."""
    certs, level, rungs = {}, 0, []
    # theta_0 is the base case and is a stipulation, as Definition (Level-1)
    # makes it. Everything above it is earned.
    certs[0] = ('lemma', 't0')
    store['t0'] = theta(0)
    names["<t0>"] = theta(0)

    for k in range(r):
        phi = theta(k)
        names.setdefault("<t%d>" % k, phi)
        C = certs[k]
        t0 = time.perf_counter()
        try:
            kernel.check(store, [], phi, C)
        except Reject as e:
            if verbose:
                print(" theta_%-2d REJECTED: %s" % (k, e))
            break
        level = k + 1
        dt = time.perf_counter() - t0
        rungs.append(dict(k=k, formula=show(phi), level=level,
                          kernel_calls=kernel.calls, seconds=round(dt, 5)))
        if verbose:
            print(" theta_%-2d %-28s certified -> Level %d"
                  % (k, show(phi), level))
        # certified necessitation: the kernel accepted C : theta_k, so
        # nec(C) is a certificate of theta_{k+1}.
        nxt = theta(k + 1)
        names["<t%d>" % (k + 1)] = nxt
        certs[k + 1] = ('nec', C)
        store["t%d" % (k + 1)] = nxt
    return level, rungs


# ============================================================================
# TARGET SUITE
```

```python
# ============================================================================
def X(n):
    return ('v', n)


TARGETS = [
    ("add_closed", ('=', ('+', ('n', 2), ('n', 2)), ('n', 4))),
    ("bnd_lt", ('allb', 'x', ('n', 5), ('<', X('x'), ('n', 5)))),
    ("bnd_ex", ('exb', 'x', ('n', 9), ('=', ('*', X('x'), X('x')),
                                       ('n', 16)))),
    ("ex_solve", ('ex', 'x', ('=', ('+', X('x'), ('n', 3)), ('n', 7)))),
    ("ex_square", ('ex', 'x', ('=', ('*', X('x'), X('x')), ('n', 49)))),
    ("ex_big", ('ex', 'x', ('=', ('+', X('x'), ('n', 1)), ('n', 18)))),
    ("all_succ", ('all', 'x', ('<', X('x'), ('s', X('x'))))),
    ("all_le", ('all', 'x', ('<=', X('x'), X('x')))),
    ("all_add0", ('all', 'x', ('=', ('+', X('x'), ('n', 0)), X('x')))),
    ("unbounded", ('all', 'x', ('ex', 'y', ('<', X('x'), X('y'))))),
    ("succ_exists", ('all', 'x', ('ex', 'y', ('=', X('y'),
                                       ('s', X('x')))))),
    ("min_exists", ('ex', 'x', ('all', 'y', ('<=', X('x'), X('y'))))),
]


def run_suite(d, w, verbose=True):
    kernel = Kernel(TAG, {})
    store = {}
    S = Search(kernel, store, d, w)
    rows, solved = [], 0
    for name, phi in TARGETS:
        t0 = time.perf_counter()
        S.nodes = 0
        C = S.prove(phi)
        dt = time.perf_counter() - t0
        verdict, ok = "", False
        if C is not None:
            try:
                kernel.check(store, [], phi, C)
                ok, verdict = True, "ok"
                store[name] = phi
            except Reject as e:
                verdict = "REJECTED: %s" % e
        solved += ok
        rows.append(dict(target=name, tier=tier_name(phi),
                         formula=show(phi), proved=C is not None,
                         certified=ok, verdict=verdict,
                         nodes=S.nodes, seconds=round(dt, 4)))
        if verbose:
            print(" %-12s %-7s %-34s %s"
                  % (name, tier_name(phi), show(phi)[:34],
                     ("CERT ok, %d nodes" % S.nodes) if ok else
                     (verdict or "not proved")))
    return solved, rows, kernel


def tier_profile(rows):
    prof = defaultdict(lambda: [0, 0])
    for r in rows:
        prof[r["tier"]][1] += 1
        if r["certified"]:
            prof[r["tier"]][0] += 1
    return {k: dict(certified=v[0], targets=v[1])
            for k, v in sorted(prof.items())}


# ============================================================================
def cmd_run(a):
    print("=" * 74)
    print(" ASCENT d=%d (depth) w=%d (witness) r=%d (reflect)"
          % (a.depth, a.witness, a.reflect))
    print("=" * 74)
    print("\n arithmetic targets\n")
    solved, rows, kernel = run_suite(a.depth, a.witness)

```

```
    print("\n reflection ladder\n")
    names = {}
    store2 = {}
    K2 = Kernel(TAG, names)
    level, rungs = climb(K2, store2, names, a.reflect)

    prof = tier_profile(rows)
    highest = max([r["tier"] for r in rows if r["certified"]],
                  default="none", key=lambda s: (s != "none", s))
    tiers_hit = sorted({r["tier"] for r in rows if r["certified"]})

    print("\n" + "-" * 74)
    print(" certified %d/%d targets | tiers reached: %s"
          % (solved, len(rows), ", ".join(tiers_hit) or "none"))
    print(" lev(A) = %d (each rung kernel-certified; r is the only cap)"
          % level)
    print(" kernel calls: %d suite + %d ladder"
          % (kernel.calls, K2.calls))
    print("-" * 74)

    os.makedirs(a.out, exist_ok=True)
    with open(os.path.join(a.out, "ascent_d%d_w%d_r%d.json"
                           % (a.depth, a.witness, a.reflect)), "w") as f:
        json.dump(dict(params=dict(d=a.depth, w=a.witness, r=a.reflect),
                       certified=solved, targets=len(rows),
                       tier_profile=prof, tiers_reached=tiers_hit,
                       lev=level, rows=rows, rungs=rungs), f, indent=2)
    return 0


def cmd_grid(a):
    """Proposition 2, empirically: sweep the knobs and show what moves."""
    print("=" * 74)
    print(" PARAMETER SWEEP -- what each knob buys")
    print("=" * 74)
    print("\n %-4s %-4s %-4s %-10s %-8s %-22s %s"
          % ("d", "w", "r", "certified", "lev", "tiers reached", "nodes"))
    print(" " + "-" * 70)
    out = []
    for d in a.depths:
        for w in a.witnesses:
            for r in a.reflects:
                solved, rows, kernel = run_suite(d, w, verbose=False)
                names, store2 = {}, {}
                K2 = Kernel(TAG, names)
                level, _ = climb(K2, store2, names, r, verbose=False)
                tiers = sorted({x["tier"] for x in rows if x["certified"]})
                nodes = sum(x["nodes"] for x in rows)
                print(" %-4d %-4d %-4d %-10s %-8d %-22s %s"
                      % (d, w, r, "%d/%d" % (solved, len(rows)), level,
                         ",".join(tiers) or "-", f"{nodes:,}"))
                out.append(dict(d=d, w=w, r=r, certified=solved,
                                targets=len(rows), lev=level,
                                tiers=tiers, nodes=nodes))
    os.makedirs(a.out, exist_ok=True)
    with open(os.path.join(a.out, "ascent_grid.json"), "w") as f:
        json.dump(out, f, indent=2)

    lev_by_r = defaultdict(set)
    cert_by_dw = defaultdict(set)
    for o in out:
        lev_by_r[o["r"]].add(o["lev"])
        cert_by_dw[(o["d"], o["w"])].add(o["certified"])
    ind_r = all(len(v) == 1 for v in lev_by_r.values())
    ind_dw = all(len(v) == 1 for v in cert_by_dw.values())
    print("\n lev depends only on r : %s" % ("YES" if ind_r else "NO"))
    print(" certified count depends only on (d,w) : %s"
          % ("YES" if ind_dw else "NO"))
    print(" -> Proposition 2 holds on this grid" if (ind_r and ind_dw)
          else " -> Proposition 2 FAILS on this grid")
    return 0


```

```
def cmd_soundness(a):
    """Hand the kernel certificates that must be rejected."""
    print("=" * 74)
    print(" KERNEL SOUNDNESS PROBES -- each of these MUST be rejected")
    print("=" * 74 + "\n")
    names = {"<f>": ('=', ('n', 0), ('n', 1))}
    K = Kernel(TAG, names)
    store = {}
    bad = [
        ("false by calc", ('=', ('n', 0), ('n', 1)), ('calc',)),
        ("lemma that is not stored",
         ('=', ('n', 0), ('n', 1)), ('lemma', 'nope')),
        ("nec for an unproved formula",
         ('pr', TAG, "<f>"), ('nec', ('calc',))),
        ("nec with the wrong tag",
         ('pr', "<Other>", "<f>"), ('nec', ('calc',))),
        ("gen from a single instance",
         ('all', 'x', ('=', X('x'), ('n', 0))),
         ('gen', 'z', ('calc',))),
        ("gen with a captured eigenvariable",
         ('all', 'x', ('=', X('x'), ('c', 'k'))),
         ('gen', 'k', ('calc',))),
        ("wit with no witness",
         ('ex', 'x', ('<', ('s', X('x')), X('x'))), ('wit', ('n', 3),
                                                     ('calc',))),
        ("induction without a base",
         ('all', 'x', ('<', X('x'), ('n', 0))),
         ('ind', ('calc',), ('calc',))),
    ]
    caught = 0
    for label, phi, C in bad:
        try:
            K.check(store, [], phi, C)
            print(" %-34s ACCEPTED <-- UNSOUND" % label)
        except Reject as e:
            caught += 1
            print(" %-34s rejected (%s)" % (label, str(e)[:30]))
    print("\n %d/%d rejected%s" % (caught, len(bad),
                                  "" if caught == len(bad)
                                  else " <-- KERNEL IS UNSOUND"))
    return 0 if caught == len(bad) else 1


def main():
    ap = argparse.ArgumentParser(
        description=__doc__,
        formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("-d", "--depth", type=int, default=3)
    ap.add_argument("-w", "--witness", type=int, default=10)
    ap.add_argument("-r", "--reflect", type=int, default=4)
    ap.add_argument("--out", default="results_ascent")
    ap.add_argument("--grid", action="store_true")
    ap.add_argument("--soundness", action="store_true")
    ap.add_argument("--depths", type=int, nargs="+", default=[1, 2, 3, 4])
    ap.add_argument("--witnesses", type=int, nargs="+", default=[1, 5, 20])
    ap.add_argument("--reflects", type=int, nargs="+", default=[1, 3, 8])
    a = ap.parse_args()
    if a.soundness:
        return cmd_soundness(a)
    if a.grid:
        return cmd_grid(a)
    return cmd_run(a)


if __name__ == "__main__":
    sys.exit(main())
```

C Ascent 2 — staged regimes

```
#!/usr/bin/env python3
r"""
ASCENT 2 -- staged regimes. Change, test, repeat.
```

```
4
5   ascent.py is frozen as the note's appendix. This file evolves it in stages,
6   each stage gated by a flag so every earlier stage stays measurable on the
7   same suite. The point is the DELTA, not the final number.
8
9       R0 base the rules ascent.py ships with
10      R1 +neg bot, notI, raa, absurd, orI, orE, exE, cut, contra, clash
11      R2 +pool every certified theorem is carried into the store, so a
12                  later target can cite it in ONE node
13      R3 +rank hand-written candidate ordering -- the baseline any
14                  learned policy has to beat
15
16  Knobs, unchanged in meaning: -t depth, -u witness bound, -v reflection.
17
18  WHAT EACH STAGE IS SUPPOSED TO SHOW
19  -----------------------------------
20  R1 exists because P != NP is a NEGATION and the base rule set has no rule
21  that introduces one. No setting of t, u, v repairs that: parameters bound
22  the search, they do not enlarge the calculus. The negated targets in the
23  suite must fail at R0 and pass at R1, or the stage did nothing.
24
25  R2 is the "learn all the tier-2 theorems and their proofs" question made
26  measurable. Citing a stored lemma costs one node no matter how expensive
27  the lemma was, which is cut, which is the only mechanism here that changes
28  proof LENGTH rather than search time. The lemma-dependent targets must fail
29  at R0/R1 and pass at R2.
30
31  R3 is a policy, not a rule. It can only reorder what R0-R2 already made
32  reachable, so it must move NODES and not COVERAGE. If R3 changes coverage,
33  something is wrong with R0-R2's search, not with the heuristic.
34
35  SOUNDNESS IS RE-CHECKED AT EVERY STAGE. Adding rules is exactly where a
36  kernel breaks, and a coverage gain bought with an unsound rule is worse than
37  no gain. Every regime runs the probe battery before it runs the suite.
38
39  Brian Tenneson. Implementation by Claude (Anthropic).
40  """
41  from __future__ import annotations
42  import argparse, itertools, json, os, sys, time
43  from collections import defaultdict
44
45  sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
46  sys.setrecursionlimit(100000)
47
48  from ascent import (sub, teval, poly, padd, pneg, show,
49                      tier_name, occurs_sym, X) # noqa: E402
50
51  BOT = ('bot',)
52  TAG = "<Ascent2>"
53  _eigen = itertools.count(1)
54
55
56  # ============================================================================
57  # DECISION PROCEDURE (extended)
58  # ============================================================================
59  def always_pos(d):
60      """Is the polynomial d strictly positive for every assignment over N?
61
62      Sufficient: every non-constant coefficient is non-negative and the
63      constant term is positive. Sound, incomplete."""
64      return all(c > 0 for m, c in d.items() if m != ()) and d.get((), 0) > 0
65
66
67  def always_nonneg(d):
68      return all(c > 0 for m, c in d.items() if m != ()) and d.get((), 0) >= 0
69
70
71  def decide(p, fuel=10000):
72      """Three-valued. Extends ascent.decide with two things R1 needs:
73
74        * bot is False;
75        * an equality is FALSE when one side strictly dominates the other over
76          N -- so Sx = 0 is refutable for symbolic x, which is what makes
```

```
        ~(Ex. Sx = 0) reachable at all."""
    k = p[0]
    if k == 'bot':
        return False
    if k in ('=', '<', '<='):
        pa, pb = poly(p[1]), poly(p[2])
        d = padd(pb, pneg(pa)) # rhs - lhs
        dn = padd(pa, pneg(pb)) # lhs - rhs
        if k == '=':
            if pa == pb:
                return True
            if always_pos(d) or always_pos(dn):
                return False # one side strictly dominates
            if all(m == () for m in d):
                return False
            return None
        if k == '<':
            if always_pos(d):
                return True
            if always_nonneg(dn):
                return False # lhs >= rhs always
            return None
        if always_nonneg(d):
            return True
        if always_pos(dn):
            return False
        return None
    if k == 'not':
        v = decide(p[1], fuel)
        return None if v is None else not v
    if k in ('and', 'or', 'imp'):
        a, b = decide(p[1], fuel), decide(p[2], fuel)
        if k == 'and':
            if a is False or b is False:
                return False
            return True if (a and b) else None
        if k == 'or':
            if a is True or b is True:
                return True
            return False if (a is False and b is False) else None
        if a is False or b is True:
            return True
        return False if (a is True and b is False) else None
    if k in ('allb', 'exb'):
        n = teval(p[2])
        if n is None or n > fuel:
            return None
        for i in range(n):
            v = decide(sub(p[3], p[1], ('n', i)), fuel)
            if v is None:
                return None
            if k == 'allb' and not v:
                return False
            if k == 'exb' and v:
                return True
        return k == 'allb'
    return None


# ============================================================================
# KERNEL
# ============================================================================
class Reject(Exception):
    pass


BASE_RULES = {'calc', 'hyp', 'lemma', 'impI', 'impE', 'andI', 'andE',
              'gen', 'inst', 'wit', 'allbI', 'exbI', 'ind', 'nec'}
NEG_RULES = {'notI', 'raa', 'absurd', 'orI', 'orE', 'exE', 'cut',
             'contra', 'clash'}


class Kernel:
```

```
    def __init__(self, tag, names, rules=None):
        self.tag = tag
        self.names = names
        self.rules = set(rules) if rules else set(BASE_RULES)
        self.calls = 0

    def check(self, store, ctx, phi, C):
        self.calls += 1
        if not isinstance(C, tuple) or not C:
            raise Reject("malformed certificate")
        k = C[0]
        if k not in self.rules:
            raise Reject("rule %r not enabled in this regime" % (k,))

        if k == 'calc':
            if decide(phi) is not True:
                raise Reject("calc: %s is not decided true" % show(phi))
            return True
        if k == 'hyp':
            if not (0 <= C[1] < len(ctx)) or ctx[C[1]] != phi:
                raise Reject("hyp: not in context")
            return True
        if k == 'lemma':
            if store.get(C[1]) != phi:
                raise Reject("lemma %s is not %s" % (C[1], show(phi)))
            return True
        if k == 'impI':
            if phi[0] != 'imp':
                raise Reject("impI: goal is not an implication")
            return self.check(store, ctx + [phi[1]], phi[2], C[1])
        if k == 'impE':
            self.check(store, ctx, ('imp', C[1], phi), C[2])
            return self.check(store, ctx, C[1], C[3])
        if k == 'andI':
            if phi[0] != 'and':
                raise Reject("andI: goal is not a conjunction")
            self.check(store, ctx, phi[1], C[1])
            return self.check(store, ctx, phi[2], C[2])
        if k == 'andE':
            i, conj = C[1], C[2]
            if conj[0] != 'and' or conj[1 + i] != phi:
                raise Reject("andE: projection mismatch")
            return self.check(store, ctx, conj, C[3])
        if k == 'gen':
            if phi[0] != 'all':
                raise Reject("gen: goal is not universal")
            nm = C[1]
            if occurs_sym(nm, phi) or any(occurs_sym(nm, h) for h in ctx) \
                    or any(occurs_sym(nm, f) for f in store.values()):
                raise Reject("gen: eigenvariable %s is not fresh" % nm)
            return self.check(store, ctx, sub(phi[2], phi[1], ('c', nm)),
                              C[2])
        if k == 'inst':
            t, univ = C[1], C[2]
            if univ[0] != 'all' or sub(univ[2], univ[1], t) != phi:
                raise Reject("inst: instance mismatch")
            return self.check(store, ctx, univ, C[3])
        if k == 'wit':
            if phi[0] != 'ex':
                raise Reject("wit: goal is not existential")
            return self.check(store, ctx, sub(phi[2], phi[1], C[1]), C[2])
        if k == 'allbI':
            if phi[0] != 'allb':
                raise Reject("allbI: goal is not bounded-universal")
            n = teval(phi[2])
            if n is None or n != len(C[1]):
                raise Reject("allbI: wrong number of instances")
            for i, Ci in enumerate(C[1]):
                self.check(store, ctx, sub(phi[3], phi[1], ('n', i)), Ci)
            return True
        if k == 'exbI':
            if phi[0] != 'exb':
                raise Reject("exbI: goal is not bounded-existential")
```

```
            n = teval(phi[2])
            if n is None or not (0 <= C[1] < n):
                raise Reject("exbI: index out of range")
            return self.check(store, ctx, sub(phi[3], phi[1], ('n', C[1])),
                              C[2])
        if k == 'ind':
            if phi[0] != 'all':
                raise Reject("ind: goal is not universal")
            x, psi = phi[1], phi[2]
            self.check(store, ctx, sub(psi, x, ('n', 0)), C[1])
            step = ('all', x, ('imp', psi, sub(psi, x, ('s', ('v', x)))))
            return self.check(store, ctx, step, C[2])
        if k == 'nec':
            if phi[0] != 'pr' or phi[1] != self.tag:
                raise Reject("nec: goal is not Pr(<A>, -)")
            psi = self.names.get(phi[2])
            if psi is None:
                raise Reject("nec: %s names no formula" % phi[2])
            return self.check(store, [], psi, C[1])

        # ---------------- R1 ----------------------------------------
        if k == 'contra':
            # a decidably FALSE hypothesis entails anything
            i = C[1]
            if not (0 <= i < len(ctx)) or decide(ctx[i]) is not False:
                raise Reject("contra: ctx[%r] is not decidably false" % (i,))
            return True
        if k == 'clash':
            i, j = C[1], C[2]
            if not (0 <= i < len(ctx) and 0 <= j < len(ctx)):
                raise Reject("clash: index out of range")
            if ctx[i] != ('not', ctx[j]):
                raise Reject("clash: ctx[%d] is not the negation of ctx[%d]"
                             % (i, j))
            return True
        if k == 'notI':
            if phi[0] != 'not':
                raise Reject("notI: goal is not a negation")
            return self.check(store, ctx + [phi[1]], BOT, C[1])
        if k == 'raa':
            # classical reductio
            return self.check(store, ctx + [('not', phi)], BOT, C[1])
        if k == 'absurd':
            A = C[1]
            self.check(store, ctx, A, C[2])
            return self.check(store, ctx, ('not', A), C[3])
        if k == 'orI':
            if phi[0] != 'or':
                raise Reject("orI: goal is not a disjunction")
            if C[1] not in (0, 1):
                raise Reject("orI: bad side")
            return self.check(store, ctx, phi[1 + C[1]], C[2])
        if k == 'orE':
            disj = C[1]
            if disj[0] != 'or':
                raise Reject("orE: not a disjunction")
            self.check(store, ctx, disj, C[2])
            self.check(store, ctx + [disj[1]], phi, C[3])
            return self.check(store, ctx + [disj[2]], phi, C[4])
        if k == 'exE':
            exphi, nm = C[1], C[2]
            if exphi[0] != 'ex':
                raise Reject("exE: not an existential")
            # The STORE must be checked as well as the goal and the context.
            # Omitting it is a genuine unsoundness, not a technicality: if the
            # store holds ~(w = 0) and exE may re-bind w while opening the
            # true statement Ex. x = 0, the opened hypothesis w = 0 clashes
            # with the lemma and bot follows from a consistent store. The
            # kernel must not rely on the search's habit of generating fresh
            # names -- that is precisely what a trusted kernel is for.
            if occurs_sym(nm, phi) or occurs_sym(nm, exphi) \
                   or any(occurs_sym(nm, h) for h in ctx) \
                   or any(occurs_sym(nm, f) for f in store.values()):
```

```
                raise Reject("exE: witness constant %s is not fresh" % nm)
            self.check(store, ctx, exphi, C[3])
            return self.check(store,
                              ctx + [sub(exphi[2], exphi[1], ('c', nm))],
                              phi, C[4])
        if k == 'cut':
            A = C[1]
            self.check(store, ctx, A, C[2])
            return self.check(store, ctx + [A], phi, C[3])

        raise Reject("unknown certificate form %r" % (k,))


# ============================================================================
# SEARCH
# ============================================================================
class Search:
    def __init__(self, kernel, store, t, u, neg=False, rank=False):
        self.K = kernel
        self.store = store
        self.t = t
        self.u = u
        self.neg = neg
        self.rank = rank
        self.nodes = 0

    # -- candidate witness terms ----------------------------------------
    def witnesses(self, phi, ctx, mode='wit'):
        # consts(), not syms(): syms() also returns BOUND VARIABLE names, so
        # this loop used to manufacture candidates like ('c','y') naming a
        # constant that does not exist, for every binder in the goal. Not
        # unsound -- the kernel rejects them -- just wasted expansions.
        out = [('n', n) for n in range(self.u + 1)]
        inscope = sorted(consts(phi) | {s for h in ctx for s in consts(h)})
        for s in inscope:
            base = ('c', s)
            out.append(base)
            tt = base
            for _ in range(min(self.u, 3)):
                tt = ('s', tt)
                out.append(tt)
            out.append(('*', ('n', 2), base))
        if self.rank:
            out.sort(key=lambda tm: self._score(tm, phi, mode))
        return out

    def _score(self, tm, phi, mode):
        """R3 heuristic -- pure policy, reorders only, admits nothing new.

        It has to be CONTEXT-SENSITIVE, and discovering that is the entire
        value of hand-building a baseline before reaching for a model. There
        are three regimes, not one, and the first version of this function
        collapsed them into a single rule and made the suite SLOWER:

          refuting a universal hypothesis -- knocked down by a small
            numeral almost every time, so size ascending;

          witnessing under a symbol in scope -- the goal sits under a gen,
            the witness is nearly always a term in the eigenconstant, so
            overlap descending (this is what buys pi3 and pi4);

          witnessing a CLOSED goal -- no eigenconstant exists to build
            from, so a symbolic term is dead weight; numerals first.

        Sorting all three the second way cost +50 nodes on min_exists alone,
        whose witness is the numeral 0. A learned policy will have to
        rediscover this split; the point of the baseline is that we now know
        what it has to beat, and why."""
        size = term_size(tm)
        if mode == 'refute':
            return (size, 0)
        goal_c = consts(phi)
        if not goal_c:
```

```
            return (0 if tm[0] == 'n' else 1, size)
        return (-len(tconsts(tm, set()) & goal_c), size)

    # -- main ------------------------------------------------------------
    def prove(self, phi, depth=None, ctx=()):
        if depth is None:
            depth = self.t
        self.nodes += 1
        if depth < 0 or self.nodes > 200000:
            return None
        ctx = tuple(ctx)

        if decide(phi) is True:
            return ('calc',)
        for i, h in enumerate(ctx):
            if h == phi:
                return ('hyp', i)
        for nm, f in self.store.items():
            if f == phi:
                return ('lemma', nm)
        if depth == 0:
            return None

        k = phi[0]

        if k == 'bot' and self.neg:
            return self.prove_bot(depth, ctx)

        if k == 'imp':
            c = self.prove(phi[2], depth - 1, ctx + (phi[1],))
            return ('impI', c) if c else None
        if k == 'and':
            c1 = self.prove(phi[1], depth - 1, ctx)
            if not c1:
                return None
            c2 = self.prove(phi[2], depth - 1, ctx)
            return ('andI', c1, c2) if c2 else None
        if k == 'or' and self.neg:
            for i in (0, 1):
                c = self.prove(phi[1 + i], depth - 1, ctx)
                if c:
                    return ('orI', i, c)
            return None
        if k == 'not' and self.neg:
            c = self.prove_bot(depth - 1, ctx + (phi[1],))
            return ('notI', c) if c else None
        if k == 'exb':
            n = teval(phi[2])
            if n is None:
                return None
            for i in range(min(n, self.u)):
                c = self.prove(sub(phi[3], phi[1], ('n', i)), depth - 1, ctx)
                if c:
                    return ('exbI', i, c)
            return None
        if k == 'allb':
            n = teval(phi[2])
            if n is None or n > self.u:
                return None
            cs = []
            for i in range(n):
                c = self.prove(sub(phi[3], phi[1], ('n', i)), depth - 1, ctx)
                if not c:
                    return None
                cs.append(c)
            return ('allbI', cs)
        if k == 'ex':
            for tm in self.witnesses(phi, ctx):
                c = self.prove(sub(phi[2], phi[1], tm), depth - 1, ctx)
                if c:
                    return ('wit', tm, c)
            return None
        if k == 'all':
```

```
            nm = "e%d" % next(_eigen)
            c = self.prove(sub(phi[2], phi[1], ('c', nm)), depth - 1, ctx)
            if c:
                return ('gen', nm, c)
            x, psi = phi[1], phi[2]
            c0 = self.prove(sub(psi, x, ('n', 0)), depth - 1, ctx)
            if c0:
                step = ('all', x, ('imp', psi, sub(psi, x, ('s', ('v', x)))))
                cs = self.prove(step, depth - 1, ctx)
                if cs:
                    return ('ind', c0, cs)
            return None

        # atoms: last resort, classical reductio
        if self.neg and depth >= 2:
            c = self.prove_bot(depth - 2, ctx + (('not', phi),))
            if c:
                return ('raa', c)
        return None

    # -- deriving absurdity --------------------------------------------
    def prove_bot(self, depth, ctx):
        self.nodes += 1
        if depth < 0 or self.nodes > 200000:
            return None
        ctx = tuple(ctx)

        # 1. a hypothesis that is outright false
        for i, h in enumerate(ctx):
            if decide(h) is False:
                return ('contra', i)
        # 2. an explicit contradictory pair
        for i, h in enumerate(ctx):
            if h[0] == 'not':
                for j, g in enumerate(ctx):
                    if h[1] == g:
                        return ('clash', i, j)
        if depth == 0:
            return None

        # 3. instantiate a universal hypothesis and look for falsity
        for i, h in enumerate(ctx):
            if h[0] != 'all':
                continue
            for tm in self.witnesses(h, ctx, mode='refute'):
                inst = sub(h[2], h[1], tm)
                if decide(inst) is False:
                    return ('cut', inst,
                            ('inst', tm, h, ('hyp', i)),
                            ('contra', len(ctx)))
        # 4. open an existential hypothesis and recurse
        for i, h in enumerate(ctx):
            if h[0] != 'ex':
                continue
            nm = "w%d" % next(_eigen)
            body = sub(h[2], h[1], ('c', nm))
            c = self.prove_bot(depth - 1, ctx + (body,))
            if c:
                return ('exE', h, nm, ('hyp', i), c)
        # 5. a negated hypothesis whose subject we can now prove
        for i, h in enumerate(ctx):
            if h[0] != 'not':
                continue
            c = self.prove(h[1], depth - 1, ctx)
            if c:
                return ('absurd', h[1], c, ('hyp', i))
        return None


def tconsts(t, acc):
    """Eigenconstants only -- ('c', name). NOT bound variables.

    syms() from ascent.py collects ('v', x) as well, so on a closed goal like
```

```
    Ex.Ay. x <= y it returns {'x','y'}: the binder names. Scoring witness
    overlap against those is meaningless, and it silently disabled the
    closed-goal branch of the heuristic below. What a witness can usefully
    be built from is the eigenconstants actually in scope, nothing else."""
    if t[0] == 'c':
        acc.add(t[1])
    elif t[0] == 's':
        tconsts(t[1], acc)
    elif t[0] in ('+', '*'):
        tconsts(t[1], acc)
        tconsts(t[2], acc)
    return acc


def consts(p, acc=None):
    if acc is None:
        acc = set()
    k = p[0]
    if k in ('=', '<', '<='):
        tconsts(p[1], acc)
        tconsts(p[2], acc)
    elif k == 'not':
        consts(p[1], acc)
    elif k in ('and', 'or', 'imp'):
        consts(p[1], acc)
        consts(p[2], acc)
    elif k in ('all', 'ex'):
        consts(p[2], acc)
    elif k in ('allb', 'exb'):
        tconsts(p[2], acc)
        consts(p[3], acc)
    return acc


def term_size(t):
    if t[0] in ('n', 'v', 'c'):
        return 1
    if t[0] == 's':
        return 1 + term_size(t[1])
    return 1 + term_size(t[1]) + term_size(t[2])


# ============================================================================
# TARGET SUITE
# ============================================================================
# group: what stage is supposed to unlock it
SUITE = [
    # --- base arithmetic, tier 0-2 -----------------------------------
    ("add_closed", 'base', ('=', ('+', ('n', 2), ('n', 2)), ('n', 4))),
    ("bnd_lt", 'base', ('allb', 'x', ('n', 5), ('<', X('x'), ('n', 5)))),
    ("ex_solve", 'base', ('ex', 'x', ('=', ('+', X('x'), ('n', 3)),
                                       ('n', 7)))),
    ("ex_square", 'base', ('ex', 'x', ('=', ('*', X('x'), X('x')),
                                       ('n', 49)))),
    ("all_succ", 'base', ('all', 'x', ('<', X('x'), ('s', X('x'))))),
    ("all_add0", 'base', ('all', 'x', ('=', ('+', X('x'), ('n', 0)),
                                        X('x')))),
    ("unbounded", 'base', ('all', 'x', ('ex', 'y', ('<', X('x'), X('y'))))),
    ("min_exists", 'base', ('ex', 'x', ('all', 'y', ('<=', X('x'), X('y'))))),
    # --- tier 3 and 4 ------------------------------------------------
    ("pi3", 'base', ('all', 'x', ('ex', 'y', ('all', 'z',
                    ('<', X('x'), ('+', X('y'), X('z'))))))),
    ("pi4", 'base', ('all', 'x', ('ex', 'y', ('all', 'z', ('ex', 'u',
                    ('<=', X('x'), ('+', X('y'), ('+', X('z'), X('u'))))))))),
    # --- negated: must FAIL at R0, PASS at R1 ------------------------
    ("no_pred_0", 'neg', ('not', ('ex', 'x', ('=', ('s', X('x')),
                                        ('n', 0))))),
    ("not_all_0", 'neg', ('not', ('all', 'x', ('=', X('x'), ('n', 0))))),
    ("no_self_lt", 'neg', ('not', ('ex', 'x', ('<', X('x'), X('x'))))),
    ("no_max", 'neg', ('not', ('ex', 'x', ('all', 'y',
                                        ('<', X('y'), X('x')))))),
    ("disj", 'neg', ('or', ('=', ('n', 1), ('n', 2)),
                           ('<', ('n', 1), ('n', 2)))),
```

```
    # --- lemma-dependent: must FAIL at R0/R1, PASS at R2 --------------
    # Each is a conjunction of targets proved EARLIER in the suite. From
    # scratch the andI node plus the deepest conjunct exceeds t; with the
    # pool each conjunct is a one-node citation, so the whole thing fits in
    # depth 2. That gap is cut buying proof LENGTH, which is the only
    # mechanism here that can.
    ("pool_2", 'pool', ('and',
                        ('all', 'x', ('ex', 'y', ('all', 'z',
                         ('<', X('x'), ('+', X('y'), X('z')))))),
                        ('all', 'x', ('ex', 'y', ('all', 'z',
                         ('ex', 'u', ('<=', X('x'),
                          ('+', X('y'), ('+', X('z'), X('u')))))))),
                        )),
    ("pool_3", 'pool', ('and',
                        ('all', 'x', ('ex', 'y', ('all', 'z',
                         ('ex', 'u', ('<=', X('x'),
                          ('+', X('y'), ('+', X('z'), X('u'))))))))),
                        ('and',
                         ('all', 'x', ('ex', 'y', ('all', 'z',
                          ('<', X('x'), ('+', X('y'), X('z')))))),
                         ('all', 'x', ('ex', 'y', ('<', X('x'),
                                                   X('y')))))),
    # --- out of reach at t=4 for every regime: tier 5 -----------------
    ("pi5", 'ceiling', ('all', 'x', ('ex', 'y', ('all', 'z',
                    ('ex', 'u', ('all', 'w',
                     ('<=', X('x'), ('+', X('y'), ('+', X('z'),
                      ('+', X('u'), X('w')))))))))),
]


def probes():
    """Malformed certificates that must be rejected in every regime."""
    return [
        ("false by calc", ('=', ('n', 0), ('n', 1)), ('calc',)),
        ("unstored lemma", ('=', ('n', 0), ('n', 1)), ('lemma', 'nope')),
        ("nec for unproved", ('pr', TAG, "<f>"), ('nec', ('calc',))),
        ("nec wrong tag", ('pr', "<Other>", "<f>"), ('nec', ('calc',))),
        ("gen from instance", ('all', 'x', ('=', X('x'), ('n', 0))),
         ('gen', 'z', ('calc',))),
        ("gen captured eigen", ('all', 'x', ('=', X('x'), ('c', 'k'))),
         ('gen', 'k', ('calc',))),
        ("wit with no witness",
         ('ex', 'x', ('<', ('s', X('x')), X('x'))),
         ('wit', ('n', 3), ('calc',))),
        ("ind without base", ('all', 'x', ('<', X('x'), ('n', 0))),
         ('ind', ('calc',), ('calc',))),
        # R1-specific
        ("contra on a true hyp", BOT, ('contra', 0)),
        ("clash on unrelated", BOT, ('clash', 0, 1)),
        ("exE captured witness",
         ('=', ('c', 'w'), ('n', 0)),
         ('exE', ('ex', 'x', ('=', X('x'), ('n', 0))), 'w',
          ('calc',), ('calc',))),
        ("raa proving falsehood", ('=', ('n', 0), ('n', 1)),
         ('raa', ('contra', 0))),
        # Regression probe. This one was NOT in the original battery, the
        # battery reported 12/12, and the hole was real: exE checked the goal
        # and the context for freshness but not the store, so bot followed
        # from a consistent store and the kernel could prove anything.
        # Found by audit.py, not by the battery.
        ("exE over a store constant", BOT,
         ('exE', ('ex', 'x', ('=', X('x'), ('n', 0))), 'w',
          ('wit', ('n', 0), ('calc',)),
          ('cut', ('not', ('=', ('c', 'w'), ('n', 0))),
           ('lemma', 'L'), ('clash', 1, 0)))),
    ]


def run_probes(K):
    ctx = [('=', ('n', 1), ('n', 1)), ('<', ('n', 0), ('n', 1))]
    # a NON-EMPTY store, so store-freshness probes can actually bite
    store = {'L': ('not', ('=', ('c', 'w'), ('n', 0)))}
    caught = total = 0
```

```
    detail = []
    for label, phi, C in probes():
        total += 1
        try:
            K.check(store, list(ctx), phi, C)
            detail.append((label, "ACCEPTED"))
        except Reject as e:
            caught += 1
            detail.append((label, str(e)[:38]))
    return caught, total, detail


# ============================================================================
# REGIMES
# ============================================================================
REGIMES = [
    ("R0 base", dict(neg=False, pool=False, rank=False)),
    ("R1 +neg", dict(neg=True, pool=False, rank=False)),
    ("R2 +pool", dict(neg=True, pool=True, rank=False)),
    ("R3 +rank", dict(neg=True, pool=True, rank=True)),
]


def run_regime(cfg, t, u, verbose=False):
    rules = set(BASE_RULES) | (NEG_RULES if cfg["neg"] else set())
    K = Kernel(TAG, {"<f>": ('=', ('n', 0), ('n', 1))}, rules)
    caught, total, pdetail = run_probes(K)

    store = {}
    S = Search(K, store, t, u, neg=cfg["neg"], rank=cfg["rank"])
    rows, solved = [], 0
    for name, group, phi in SUITE:
        S.nodes = 0
        t0 = time.perf_counter()
        C = S.prove(phi)
        dt = time.perf_counter() - t0
        ok = False
        note = ""
        if C is not None:
            try:
                K.check(store, [], phi, C)
                ok = True
            except Reject as e:
                note = "REJECTED: %s" % str(e)[:40]
        solved += ok
        # R2: carry the certified theorem forward as a citable lemma
        if ok and cfg["pool"]:
            store[name] = phi
        rows.append(dict(target=name, group=group, tier=tier_name(phi),
                         certified=ok, nodes=S.nodes,
                         seconds=round(dt, 4), note=note))
        if verbose:
            print(" %-12s %-6s %-7s %s %s nodes"
                  % (name, group, tier_name(phi),
                     "OK " if ok else " . ", f"{S.nodes:,}"))
    # Nodes on CERTIFIED targets is the number a policy can move. Nodes
    # burned on a target that fails is just how long exhaustion takes, and
    # no reordering shortens an exhaustive search -- pi5 costs 4,417 in
    # every regime and swamps the total if you report it undivided.
    return dict(solved=solved, rows=rows,
                nodes=sum(r["nodes"] for r in rows),
                nodes_certified=sum(r["nodes"] for r in rows
                                    if r["certified"]),
                probes_caught=caught, probes_total=total,
                probe_detail=pdetail, unsound=(caught != total))


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("-t", "--depth", type=int, default=4)
    ap.add_argument("-u", "--witness", type=int, default=12)
    ap.add_argument("-v", "--reflect", type=int, default=3)
    ap.add_argument("--out", default="results_ascent2")
```

```
    ap.add_argument("--verbose", action="store_true")
    a = ap.parse_args()

    print("=" * 78)
    print(" ASCENT 2 -- staged regimes t=%d u=%d v=%d"
          % (a.depth, a.witness, a.reflect))
    print("=" * 78)

    results, prev = [], None
    for label, cfg in REGIMES:
        r = run_regime(cfg, a.depth, a.witness, a.verbose)
        r["regime"] = label
        results.append(r)
        by_group = defaultdict(lambda: [0, 0])
        for row in r["rows"]:
            by_group[row["group"]][1] += 1
            if row["certified"]:
                by_group[row["group"]][0] += 1
        gsum = " ".join("%s %d/%d" % (g, v[0], v[1])
                        for g, v in sorted(by_group.items()))
        delta = "" if prev is None else " (%+d)" % (r["solved"] -
                                                    prev["solved"])
        ndelta = ""
        if prev is not None and prev["nodes_certified"]:
            pct = 100.0 * (prev["nodes_certified"] - r["nodes_certified"]) \
                / prev["nodes_certified"]
            ndelta = " (%+.0f%%)" % (-pct)
        print("\n %-9s certified %2d/%-2d%-7s nodes-on-certified %5s%-8s"
              " soundness %d/%d%s"
              % (label, r["solved"], len(SUITE), delta,
                 f"{r['nodes_certified']:,}", ndelta,
                 r["probes_caught"], r["probes_total"],
                 " <-- UNSOUND" if r["unsound"] else ""))
        print(" %s" % gsum)
        if prev is not None:
            gained = [x["target"] for x, y in zip(r["rows"], prev["rows"])
                      if x["certified"] and not y["certified"]]
            lost = [x["target"] for x, y in zip(r["rows"], prev["rows"])
                    if y["certified"] and not x["certified"]]
            if gained:
                print(" gained: %s" % ", ".join(gained))
            if lost:
                print(" LOST: %s <-- regression"
                      % ", ".join(lost))
        prev = r

    os.makedirs(a.out, exist_ok=True)
    with open(os.path.join(a.out, "regimes.json"), "w") as f:
        json.dump(dict(params=dict(t=a.depth, u=a.witness, v=a.reflect),
                       results=results), f, indent=2)
    print("\n wrote %s/regimes.json" % a.out)
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

D The audit

```
#!/usr/bin/env python3
r"""
audit.py -- adversarial audit of the Ascent kernels.

Three independent checks, none of which trusts the implementation's own
opinion of itself:

 1. FUZZ THE DECISION PROCEDURE. Generate random formulas over symbolic
    constants; wherever decide() commits to True or False, verify by brute
    force over every assignment in a finite box. A single mismatch is an
    unsoundness, because decide() is what the `calc` rule trusts.

 2. EXPLOIT THE FRESHNESS CONDITIONS. gen and exE both introduce a constant
```

```
   that is supposed to be arbitrary. Try to smuggle in a constant that is
   already committed elsewhere -- in the goal, the context, or the LEMMA
   STORE -- and derive something false.

 3. CERTIFY-THEN-RECHECK. Re-verify every certificate the search produces
   against a kernel built from scratch, and additionally check that the
   certified formula is true under brute-force evaluation wherever it is
   checkable. A prover that proves a falsehood must be caught here even if
   its own kernel accepted it.

   python audit.py
"""
from __future__ import annotations
import itertools, random, sys, os

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
sys.setrecursionlimit(100000)

import ascent as A1
import ascent2 as A2
from ascent import show, X

BOX = 6 # brute-force each symbol over 0..BOX-1
random.seed(20260730)


# ---------------------------------------------------------------------------
# 1. fuzz the decision procedure
# ---------------------------------------------------------------------------
def rnd_term(symbols, d=2):
    if d == 0 or random.random() < 0.4:
        if symbols and random.random() < 0.6:
            return ('c', random.choice(symbols))
        return ('n', random.randint(0, 4))
    k = random.choice(['+', '*', 's'])
    if k == 's':
        return ('s', rnd_term(symbols, d - 1))
    return (k, rnd_term(symbols, d - 1), rnd_term(symbols, d - 1))


def rnd_formula(symbols, d=2):
    if d == 0 or random.random() < 0.5:
        op = random.choice(['=', '<', '<='])
        return (op, rnd_term(symbols, 2), rnd_term(symbols, 2))
    k = random.choice(['not', 'and', 'or', 'imp'])
    if k == 'not':
        return ('not', rnd_formula(symbols, d - 1))
    return (k, rnd_formula(symbols, d - 1), rnd_formula(symbols, d - 1))


def ground_eval(p, env):
    """Truth of p under a total assignment env of constants to naturals."""
    k = p[0]
    if k == 'bot':
        return False
    if k in ('=', '<', '<='):
        a, b = term_val(p[1], env), term_val(p[2], env)
        return a == b if k == '=' else (a < b if k == '<' else a <= b)
    if k == 'not':
        return not ground_eval(p[1], env)
    if k == 'and':
        return ground_eval(p[1], env) and ground_eval(p[2], env)
    if k == 'or':
        return ground_eval(p[1], env) or ground_eval(p[2], env)
    if k == 'imp':
        return (not ground_eval(p[1], env)) or ground_eval(p[2], env)
    raise ValueError("ground_eval: %r" % (k,))


def term_val(t, env):
    k = t[0]
    if k == 'n':
        return t[1]
```

```
    if k in ('c', 'v'):
        return env[t[1]]
    if k == 's':
        return term_val(t[1], env) + 1
    if k == '+':
        return term_val(t[1], env) + term_val(t[2], env)
    return term_val(t[1], env) * term_val(t[2], env)


def fuzz_decide(decide, label, trials=6000):
    syms = ['a', 'b']
    bad = []
    committed = 0
    for _ in range(trials):
        p = rnd_formula(syms, 2)
        v = decide(p)
        if v is None:
            continue
        committed += 1
        for vals in itertools.product(range(BOX), repeat=len(syms)):
            env = dict(zip(syms, vals))
            if ground_eval(p, env) != v:
                bad.append((p, v, env))
                break
    print(" %-28s %5d/%d committed, %d WRONG"
          % (label, committed, trials, len(bad)))
    for p, v, env in bad[:4]:
        print(" claimed %-5s for %s at %s"
              % (v, show(p), env))
    return len(bad)


# ------------------------------------------------------------------------
# 2. exploit the freshness conditions
# ------------------------------------------------------------------------
def exploit_freshness():
    print("\n freshness exploits (each MUST be rejected)")
    fails = 0

    # (a) gen re-using a constant that is committed in the STORE
    K = A2.Kernel(A2.TAG, {}, set(A2.BASE_RULES) | A2.NEG_RULES)
    store = {'committed': ('=', ('c', 'k'), ('n', 0))} # k = 0 is a lemma
    goal = ('all', 'x', ('=', X('x'), ('n', 0))) # false: not all x = 0
    cert = ('gen', 'k', ('lemma', 'committed'))
    try:
        K.check(store, [], goal, cert)
        print(" gen over a store constant ACCEPTED <-- UNSOUND")
        fails += 1
    except A2.Reject as e:
        print(" gen over a store constant rejected (%s)"
              % str(e)[:34])

    # (b) exE re-using a constant that is committed in the STORE.
    # The store asserts ~(w = 0). Ex.x=0 is TRUE, so exE is entitled to
    # open it -- but only over a FRESH constant. If it may re-use w, the
    # opened hypothesis w = 0 clashes with the stored lemma and bot
    # follows from a perfectly consistent store, i.e. everything does.
    ex = ('ex', 'x', ('=', X('x'), ('n', 0)))
    Lw = ('not', ('=', ('c', 'w'), ('n', 0)))
    store2 = {'L': Lw}
    cert2 = ('exE', ex, 'w',
             ('wit', ('n', 0), ('calc',)),
             ('cut', Lw, ('lemma', 'L'), ('clash', 1, 0)))
    try:
        K.check(store2, [], A2.BOT, cert2)
        print(" exE over a store constant ACCEPTED <-- UNSOUND")
        fails += 1
    except A2.Reject as e:
        print(" exE over a store constant rejected (%s)"
              % str(e)[:34])

    # (c) exE re-using a constant already in the context
    ctx = [('=', ('c', 'w'), ('n', 3))]
```

```
    try:
        K.check({}, list(ctx), A2.BOT,
                ('exE', ex, 'w', ('wit', ('n', 0), ('calc',)),
                 ('contra', 0)))
        print(" exE over a context constant ACCEPTED <-- UNSOUND")
        fails += 1
    except A2.Reject as e:
        print(" exE over a context constant rejected (%s)"
              % str(e)[:34])
    return fails


# ---------------------------------------------------------------------------
# 3. certify-then-recheck the whole suite
# ---------------------------------------------------------------------------
def recheck_suite():
    print("\n independent re-verification of every certificate")
    bad = 0
    for label, cfg in A2.REGIMES:
        rules = set(A2.BASE_RULES) | (A2.NEG_RULES if cfg["neg"] else set())
        K = A2.Kernel(A2.TAG, {}, rules)
        store = {}
        S = A2.Search(K, store, 4, 12, neg=cfg["neg"], rank=cfg["rank"])
        n_ok = n_bad = 0
        for name, group, phi in A2.SUITE:
            C = S.prove(phi)
            if C is None:
                continue
            fresh = A2.Kernel(A2.TAG, {}, rules) # brand-new kernel
            try:
                fresh.check(store, [], phi, C)
            except A2.Reject:
                n_bad += 1
                continue
            # and is the formula actually TRUE?
            cs = sorted(A2.consts(phi))
            truth = True
            if len(cs) <= 2:
                for vals in itertools.product(range(BOX), repeat=len(cs)):
                    env = dict(zip(cs, vals))
                    try:
                        if not closed_truth(phi, env):
                            truth = False
                            break
                    except (ValueError, KeyError):
                        truth = None
                        break
            else:
                truth = None
            if truth is False:
                print(" %-10s CERTIFIED A FALSEHOOD: %s"
                      % (name, show(phi)))
                n_bad += 1
            else:
                n_ok += 1
                if cfg["pool"]:
                    store[name] = phi
        print(" %-9s %2d certificates re-verified, %d bad"
              % (label, n_ok, n_bad))
        bad += n_bad
    return bad


def closed_truth(p, env, box=BOX):
    """Brute-force truth over the box, quantifiers included."""
    k = p[0]
    if k == 'bot':
        return False
    if k in ('=', '<', '<='):
        return ground_eval(p, env)
    if k == 'not':
        return not closed_truth(p[1], env, box)
    if k == 'and':
```

```
        return closed_truth(p[1], env, box) and closed_truth(p[2], env, box)
    if k == 'or':
        return closed_truth(p[1], env, box) or closed_truth(p[2], env, box)
    if k == 'imp':
        return (not closed_truth(p[1], env, box)) \
            or closed_truth(p[2], env, box)
    if k in ('all', 'ex', 'allb', 'exb'):
        # unbounded quantifiers cannot be settled in a finite box; report
        # unknown rather than guessing
        raise ValueError("quantified")
    raise ValueError(k)


# ---------------------------------------------------------------------------
# ---------------------------------------------------------------------------
# 4. substitution and tier -- regression tests for two bugs found by audit
# ---------------------------------------------------------------------------
def check_substitution():
    """sub( Ay. x = y , x := y ) once returned Ay. y = y, capturing the free
    y. The callers happened to be safe (certificate terms are built from
    numerals and eigenconstants, a separate namespace from bound variables),
    but `ind` does substitute a variable-bearing term, so the guarantee must
    not rest on caller discipline."""
    print("\n substitution")
    bad = 0
    cases = [
        (('all', 'y', ('=', X('x'), X('y'))), 'x', X('y')),
        (('ex', 'y', ('<', X('y'), X('x'))), 'x', X('y')),
        (('all', 'y', ('ex', 'z', ('=', X('x'), ('+', X('y'), X('z'))))),
         'x', ('+', X('y'), X('z'))),
    ]
    for f, x, v in cases:
        g = A1.sub(f, x, v)
        # after substitution the incoming term's variables must still be FREE
        free_after = A1.fvars(g)
        need = A1.tvars(v)
        ok = need <= free_after
        print(" %-34s -> %-30s %s"
              % (show(f)[:34], show(g)[:30], "ok" if ok else "CAPTURED"))
        bad += 0 if ok else 1
    # shadowing must be a no-op
    f = ('all', 'x', ('=', X('x'), X('x')))
    if A1.sub(f, 'x', ('n', 3)) != f:
        print(" shadowed binder was substituted <-- WRONG")
        bad += 1
    return bad


def check_tier():
    """tier() treated `imp` as symmetric, ignoring that A -> B is ~A or B and
    the antecedent therefore sits in negative position; and it joined mixed
    leads by taking whichever had the larger subscript."""
    print("\n tier")
    P = ('all', 'x', ('<', X('x'), ('n', 1)))
    S = ('ex', 'x', ('<', X('x'), ('n', 1)))
    cases = [
        (P, "Pi0-1"), (S, "Sig0-1"), (('not', S), "Pi0-1"),
        (('not', P), "Sig0-1"),
        (('imp', P, S), "Sig0-1"),
        # (Ex.P) -> (Ex.P) is (Ax.~P) or (Ex.P): mixed leads at level 1, so
        # Delta-0-2 SYNTACTICALLY, even though it is a tautology and hence
        # semantically Delta-0-0. tier() computes the syntactic class, which
        # is the only non-degenerate one for closed sentences -- every true
        # sentence is semantically equivalent to 0 = 0. This case is in the
        # battery because the author's first expectation for it was wrong.
        (('imp', S, S), "Del0-2"),
        (('and', S, P), "Del0-2"),
        (('or', S, P), "Del0-2"),
        (('and', P, P), "Pi0-1"),
        (('all', 'x', ('ex', 'y', ('<', X('x'), X('y')))), "Pi0-2"),
        (('not', ('ex', 'x', ('all', 'y', ('<', X('y'), X('x'))))), "Pi0-2"),
    ]
    bad = 0
```

```
    for f, expect in cases:
        got = A1.tier_name(f)
        if got != expect:
            print(" %-40s %-8s expected %s"
                  % (show(f)[:40], got, expect))
            bad += 1
    print(" %d/%d classifications correct" % (len(cases) - bad,
                                              len(cases)))
    return bad


def main():
    print("=" * 74)
    print(" ASCENT AUDIT")
    print("=" * 74)
    print("\n fuzzing the decision procedure (%d assignments per formula)"
          % (BOX ** 2))
    n = 0
    n += fuzz_decide(A1.decide, "ascent.decide")
    n += fuzz_decide(A2.decide, "ascent2.decide")
    n += check_substitution()
    n += check_tier()
    n += exploit_freshness()
    n += recheck_suite()
    print("\n" + "=" * 74)
    print(" %s" % ("ALL CHECKS PASSED" if n == 0
                   else "%d PROBLEM(S) FOUND" % n))
    print("=" * 74)
    return 0 if n == 0 else 1


if __name__ == "__main__":
    sys.exit(main())
```